
Magicdog-Y1 SDK Development Documentation

发布 *1.2.3-doc2*

MagicLab

2026 年 06 月 01 日

1	关于 MagicDog-Y1	1
1.1	文档内容	1
2	SDK 概述	3
2.1	SDK 通信接口介绍	3
2.2	获取 SDK	4
3	软硬件架构	5
3.1	软件系统架构图	6
3.2	硬件系统架构图	7
4	服务介绍	9
5	快速开始	11
5.1	系统环境	11
5.2	网络环境	12
5.3	安装与编译	15
5.4	例程示例	17
5.5	Python 例程示例	18
6	机器人主控服务	21
6.1	接口定义	21
7	高层运动控制服务	27
7.1	接口定义	27
7.2	类型定义	30
7.3	枚举类型定义	30
7.4	遥控器示意图	31
7.5	高层运动控制机器人状态介绍	32

7.6 高层运动控制接口	32
8 底层运动控制服务	35
9 传感器服务	37
10 音频服务	39
11 状态监控服务	41
11.1 接口定义	41
11.2 错误码映射表	43
12 SLAM 导航控制服务	47
12.1 接口定义	47
12.2 类型定义	56
13 机器人主控服务 (Python)	59
13.1 接口定义	59
14 高层运动控制服务 (Python)	65
14.1 接口定义	65
14.2 类型定义	68
14.3 枚举类型定义	68
14.4 遥控器示意图	69
14.5 高层运动控制机器人状态介绍	69
14.6 高层运动控制接口	70
15 底层运动控制服务	73
16 传感器服务	75
17 音频服务	77
18 状态监控服务 (Python)	79
18.1 接口定义	79
18.2 错误码映射表	81
19 SLAM 导航控制服务 (Python)	85
19.1 接口定义	85
19.2 类型定义	93
20 高层运动控制示例	97
20.1 C++:	97
20.2 Python	102
20.3 完整代码示例	102
20.4 运行说明	111

21 底层运动控制示例	113
21.1 C++:	113
21.2 Python	116
21.3 完整代码示例	116
21.4 运行说明	125
22 传感器控制示例	127
22.1 C++	127
22.2 Python	130
22.3 完整代码示例	130
22.4 运行说明	134
23 语音控制示例	135
23.1 C++	135
23.2 Python	141
23.3 完整代码示例	141
23.4 运行说明	150
24 状态监控示例	151
24.1 C++	151
24.2 Python	153
24.3 完整代码示例	153
24.4 运行说明	156
25 FAQ	157
25.1 哪些能力已经开放?	157
25.2 哪些能力暂不开放?	157
25.3 图片无法显示怎么办?	157
25.4 支持无线开发吗?	157
25.5 SDK 接口调用报错 <code>Deadline Exceeded</code> 怎么办?	157
25.6 SDK 无法订阅话题数据或话题数据不生效怎么办?	158
25.7 日志出现 <code>Call of pthread_setschedparam with insufficient privileges!</code> 怎么办?	158
25.8 运行时报 <code>Invalid IP address</code> 怎么办?	158
25.9 SLAM 导航不执行怎么办?	158

关于 MagicDog-Y1

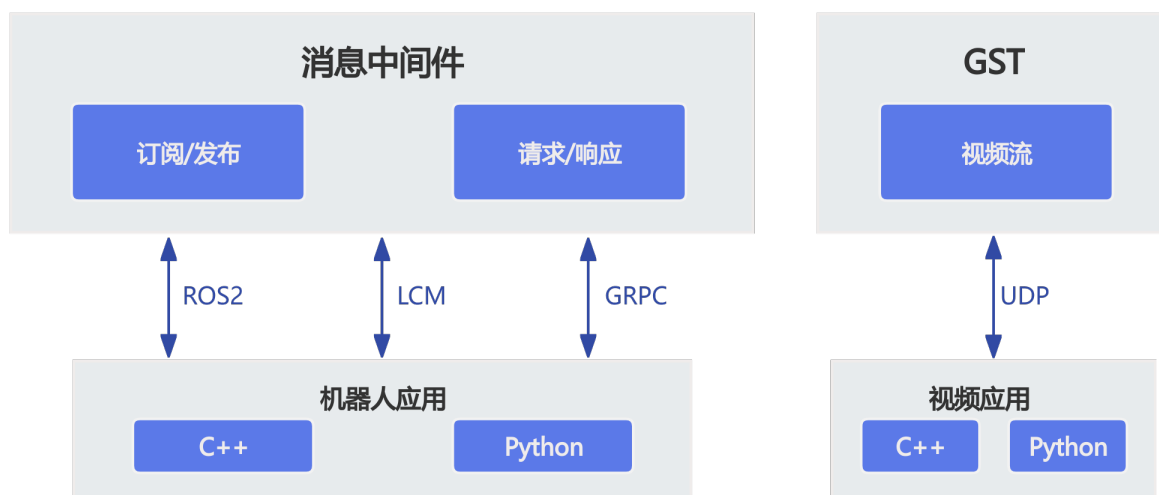
MagicDog-Y1 是面向四足机器人开发的 MagicDog 系列平台。Y1 SDK 提供机器人连接、状态监控、高层运动控制和 SLAM 导航等接口，方便开发者在应用层完成状态切换、运动模式切换、遥控控制和导航任务。

当前文档优先开放主控、高层运动、SLAM 导航和状态监控相关能力。传感器、音频和底层运动控制文档正在开发中，暂不作为公开接口说明。

1.1 文档内容

- SDK 介绍：了解通信方式、架构、服务和快速开始流程。
- C++ API：查看主控、高层运动、SLAM 导航和状态监控接口。
- Python API：查看 Python 绑定接口。
- 示例：查看已开放能力的调用示例。

2.1 SDK 通信接口介绍



- 当前软件版本暂不支持 GST 视频流传输

Y1 采用 LCM/GRPC 作为消息中间件，主要数据交互模式：话题（订阅/发布）和 RPC（请求/响应）

- **话题（订阅/发布）**：接收方订阅某个消息，发送方根据订阅列表向接收方发送消息，主要用于中高频或持续的数据交互。
- **RPC（请求/响应）**：问答模式，通过请求实现数据获取或操作。用于低频或功能切换时的数据交互。

话题和 RPC 接口的主要调用方式：函数式接口

- **函数式接口**：将 API 调用封装成函数调用，方便用户使用。

2.2 获取 SDK

magicdog-y1_sdk 是魔法原子新一代机器人 Y1 开发 SDK。SDK 将高层运动控制、底层电机控制、语音控制等接口进行封装，并提供相关的函数式接口。您可以参考我们提供的 SDK 教程，学习机器人控制，完成 z1 的二次开发；

2.2.1 SDK 下载地址：

`magicdog-y1_sdk`

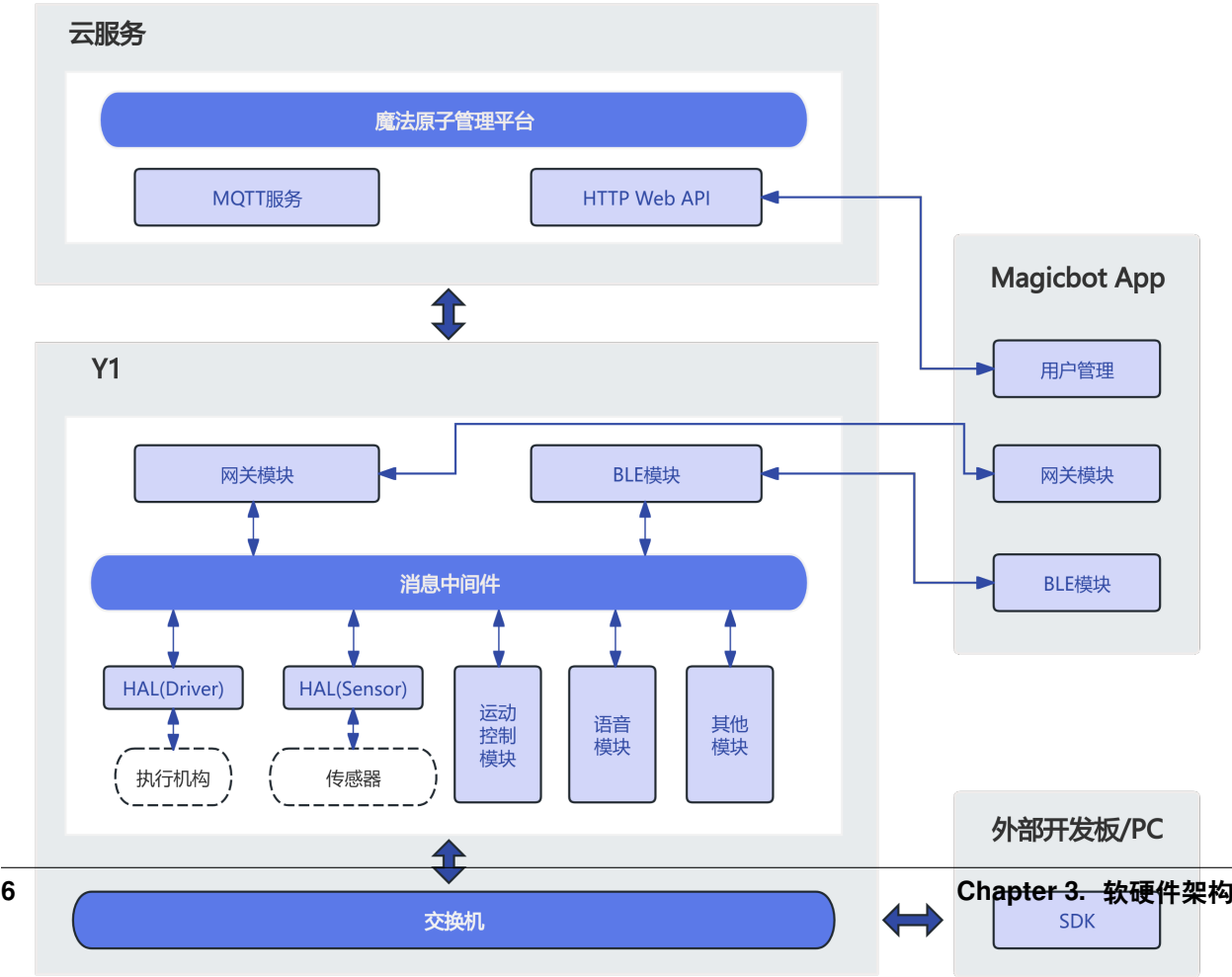
2.2.2 URDF/MJCF 下载地址：

`URDF/MJCF`

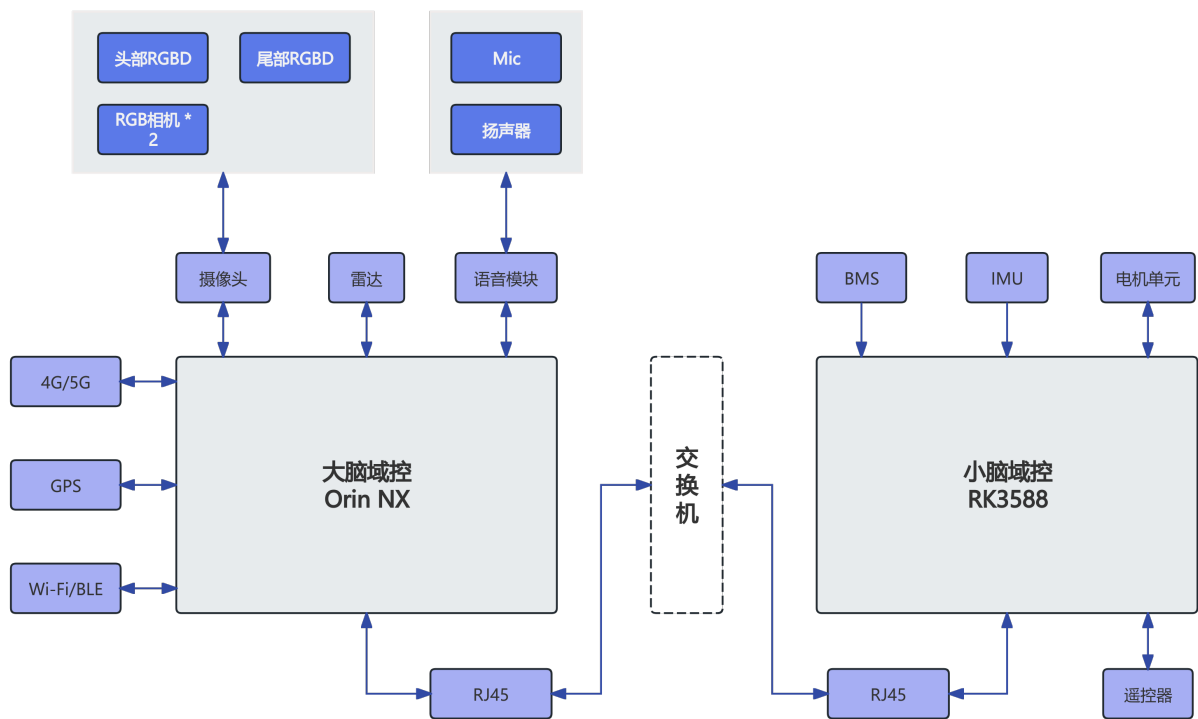
CHAPTER 3

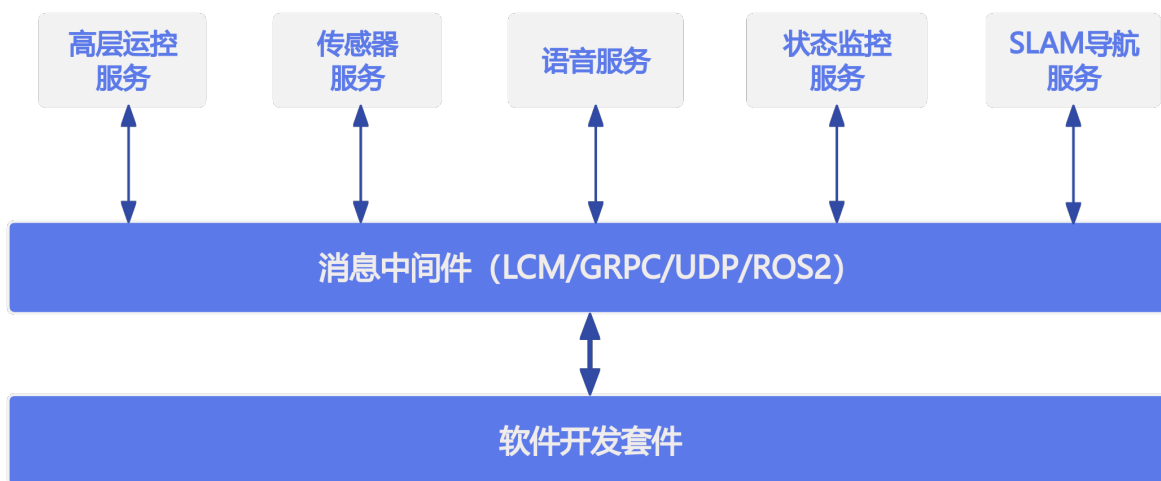
软硬件架构

3.1 软件系统架构图



3.2 硬件系统架构图





Magicdog Y1 通过消息中间件 (LCM/GRPC/UDP) 提供如下服务：

- 高层运控服务

基于 Y1 内置步态控制器，可进行步态切换、特技执行、控制姿态和速度（等价遥控器）等功能。高层运控服务使用 GRPC 进行通信。

- 底层运控服务

通过服务接口可实时获取关节、IMU 等数据，同时可实时下发关节指令控制电机。底层运控服务使用 lcm 进行通信。

- **语音服务**

通过服务接口可控制音量和语音控制。语音服务使用 `grpc/lcm` 进行通信。

- **传感器服务**

支持 lidar、rgbd、双目相机等传感器的数据订阅。传感器服务使用 `grpc` 和 `lcm` 进行通信。

- **状态监控服务**

通过服务接口可订阅机器人本体软硬件故障和 `bms` 状态。状态监控服务使用 `grpc` 进行通信。

5.1 系统环境

推荐在 ubuntu22.04 系统下开发，暂不支持 Mac/Windows 系统下开发，机器人本体 PC 运行官方服务，不支持开发；

5.1.1 开发环境要求

- GCC $\geq$ 11.4 (for Linux)
- CMake $\geq$ 3.16
- Make build system
- C++20 (minimum)
- Eigen3
- python3.10

5.1.2 第三方库依赖

```
sudo apt update
sudo apt install libgrpc++-dev libgrpc-dev liblcm-dev libyaml-cpp-dev libsystemd-dev -y
sudo apt install cmake python3 python3-dev python3-pip libeigen3-dev build-essential -y
```

5.1.3 系统配置

首先，为了实现普通用户下通信实时性，需要在/etc/security/limits.conf 文件中增加如下配置：

```
*      -      rtprio      98
```

其次，为了增加每个 socket 链接的接收缓存，需要在/etc/sysctl.conf 文件中增加如下配置，sudo sysctl -p 立即生效或重启生效：

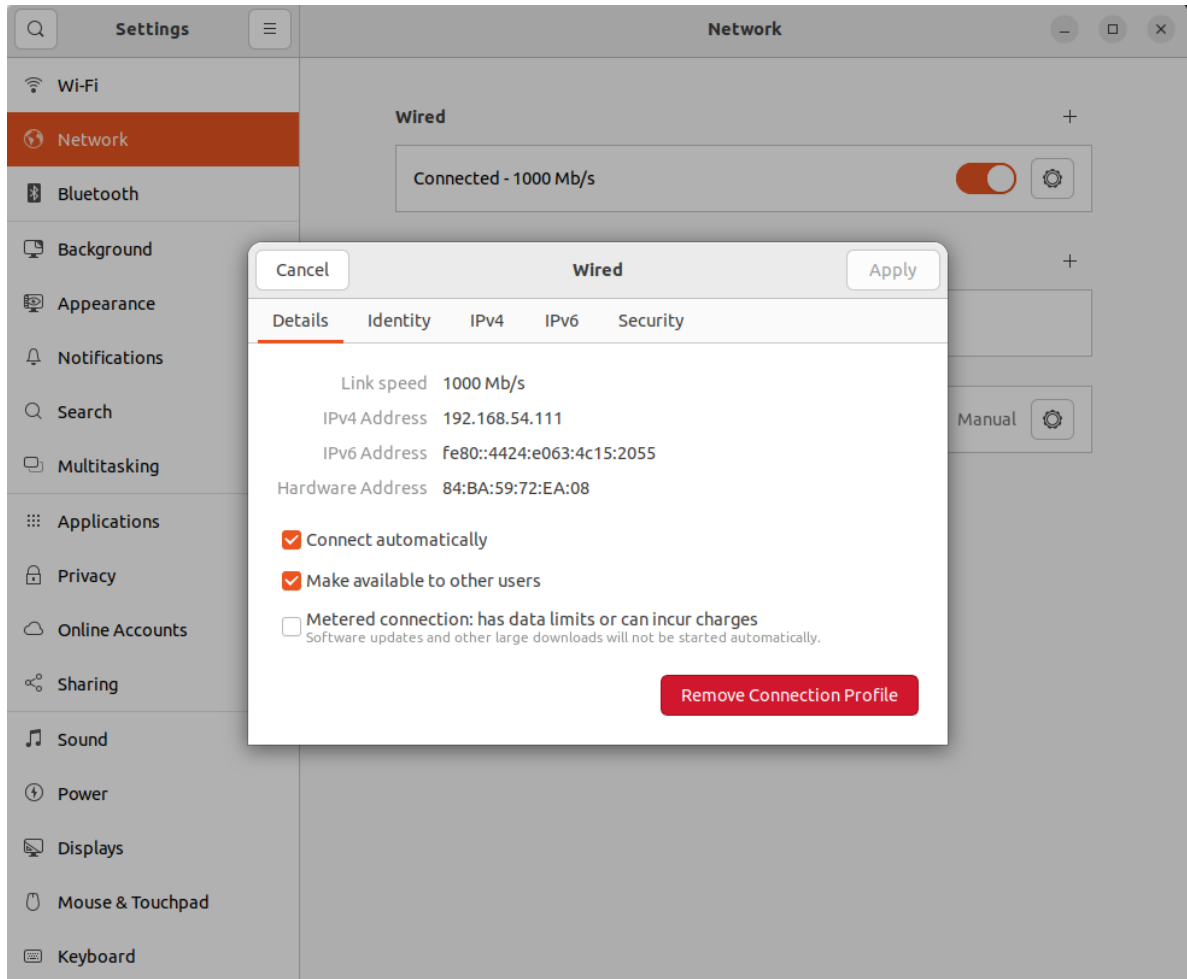
```
net.core.rmem_max=20971520
net.core.rmem_default=20971520
net.core.wmem_max=20971520
net.core.wmem_default=20971520
```

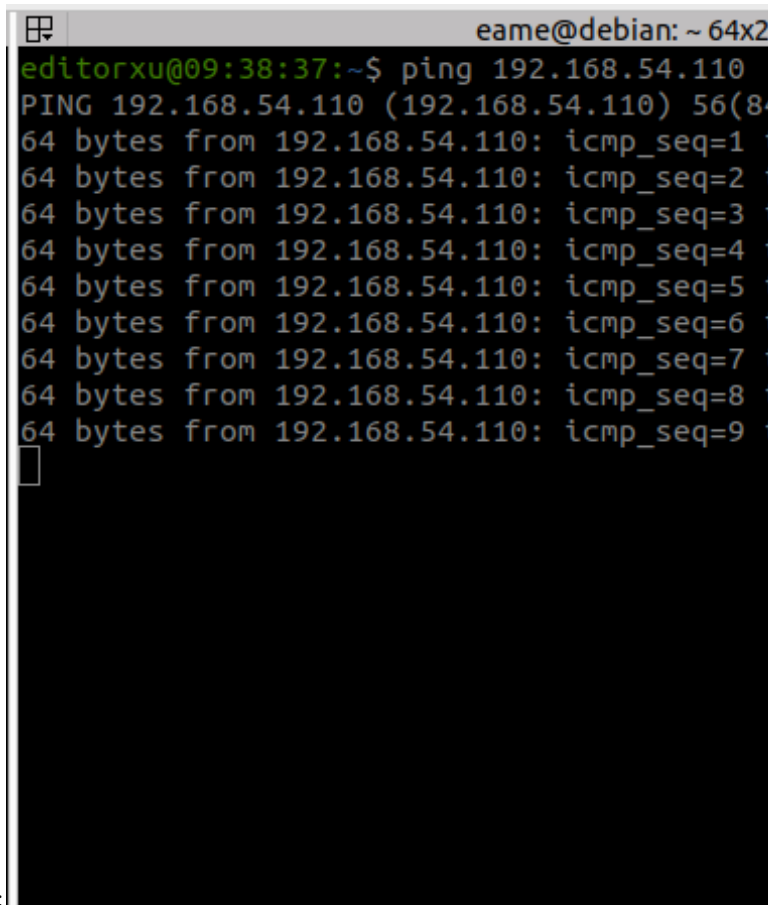
5.2 网络环境

将用户计算机与机器人交换机接入统一网络。建议新用户使用网线将用户计算机接入机器人交换机，并将与机器人通信的网卡设置在 192.168.54.X 网段下，推荐使用 192.168.54.111。有经验的用户可自行配置网络环境。

5.2.1 配置步骤

1. 用网线的一端连接机器人，另一端连接用户电脑。机器人机载电脑的 IP 地址为 192.168.54.110（小脑算力板）和 192.168.54.119（大脑算力板），所以需要将电脑 ip 设置为同一网段，建议 192.168.54.111.



A terminal window with a dark background. The title bar shows a window icon, the username 'eame@debian', and the path '~ 64x2'. The prompt is 'editorxu@09:38:37:~\$'. The command 'ping 192.168.54.110' has been entered. The output shows 'PING 192.168.54.110 (192.168.54.110) 56(84) bytes of data.' followed by nine lines of '64 bytes from 192.168.54.110: icmp_seq=1' through 'icmp_seq=9'.

```
eame@debian: ~ 64x2
editorxu@09:38:37:~$ ping 192.168.54.110
PING 192.168.54.110 (192.168.54.110) 56(84) bytes of data:
64 bytes from 192.168.54.110: icmp_seq=1 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=2 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=3 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=4 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=5 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=6 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=7 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=8 ttl=64 time=0.054 ms
64 bytes from 192.168.54.110: icmp_seq=9 ttl=64 time=0.054 ms
```

2. 为了测试通信连接是否正常,可以通过 ping 进行测试:

3. 查看 192.168.54.111 网址对应的网卡名称,通过 `ifconfig` 命令查看网卡名字,如图所示:

```
eame@debian: ~/ws/hal_driver/build/install/bin
eame@debian: ~/ws/hal_driver/build/install/bin 92x48
editorxu@11:35:09:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:a5:79:70:7c txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp0s31f6: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.54.111 netmask 255.255.255.0 broadcast 192.168.54.255
    inet6 fe80::4424:e003:4c15:2055 prefixlen 64 scopeid 0x20<link>
    ether 84:ba:59:72:ea:08 txqueuelen 1000 (Ethernet)
    RX packets 1042289 bytes 354005321 (354.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1050126 bytes 280690787 (280.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 16 memory 0x82200000-82220000

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 87371077 bytes 9065855211 (9.0 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 87371077 bytes 9065855211 (9.0 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

virbr0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
    ether 52:54:00:9e:77:df txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.29.41.197 netmask 255.255.252.0 broadcast 172.29.43.255
    inet6 fe80::4eb3:a694:2eba:ac34 prefixlen 64 scopeid 0x20<link>
    ether 5c:b4:7e:8f:79:97 txqueuelen 1000 (Ethernet)
    RX packets 26672293 bytes 13099885890 (13.0 GB)
    RX errors 0 dropped 101 overruns 0 frame 0
    TX packets 20349743 bytes 3661275993 (3.6 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

假设 SDK 二次开发 PC 与机器人链接的网口为 enp0s31f6，需要进行如下配置以便 SDK 与机器人的底层通信：

```
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev enp0s31f6
```

5.3 安装与编译

以下步骤假设工作目录为/home/magicdog/workspace

5.3.1 安装 magicdog-y1_sdk:

安装步骤: 进入 magicdog-y1_sdk 目录, 并按步骤执行如下命令, 将 sdk 安装至 /opt/magic_robotics/magic_y1_sdk 目录下, 便于二次开发程序 find_package 检索编译链接:

```
cd /home/magicdog/workspace/magicdog-y1_sdk/
mkdir build
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/opt/magic_robotics/magic_y1_sdk
make -j8
sudo make install
```

5.3.2 例程编译

cmake 方式

```
cd /home/magicbot/workspace/magicdog_sdk/
mkdir build
cd build
cmake .. -DBUILD_EXAMPLES=ON
make -j8
```

运行上述命令后, 若进度进行到 100% 且没有报错, 则意味着编译成功。

g++ 方式

使用 g++ 编译 c++ 例程文件示例如下, 其中 -I 参数后为头文件安装目录, -L 参数后为库文件安装目录。生成的可执行程序就在当前目录。

```
g++ high_level_motion_example.cpp -I /opt/magic_robotics/magic_y1_sdk/include -l magicdog_y1_
↪sdk -L /opt/magic_robotics/magic_y1_sdk/lib -o high_level_motion_example
```

5.3.3 用户模块导入 magicdog-y1_sdk

如果用户需要自己的模块中引入 magicdog_y1_sdk, 可以参考 example/cmake_example/CMakeLists.txt。

5.3.4 运行说明

1. 环境准备

按照上面步骤安装后, 库文件会被安装在 /opt/magic_robotics/magic_y1_sdk 目录。也可以直接将提供的库文件放到自定义目录, 只需将下面 export 命令中 /opt/magic_robotics/magic_y1_sdk 改为放置库文件的目录名。

```
# 设置环境变量
export PYTHONPATH=/opt/magic_robotics/magicbot_y1_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicbot_y1_sdk/lib:$LD_LIBRARY_PATH
```

2. 运行示例

```
# C++
./high_level_motion_example
# Python
python3 high_level_motion_example.py
```

5.4 例程示例

magicdog-y1_sdk/build 目录中：

- 语音示例：
 - audio_example
- 传感器示例：
 - sensor_example
- 状态监控示例：
 - monitor_example
- 底层运控示例：
 - low_level_motion_example
- 高层运控示例：
 - high_level_motion_example

Notice: 其中，高层运控示例中几种示例有先后依赖顺序（锁定站立-> 平衡站立-> 执行特技/遥控器控制）

5.4.1 进入调试模式：

按照操作流程，确保机器人进入调试模式

5.4.2 运行例程

进入 magicdog-y1_sdk/build 目录，执行如下命令：

```
cd /home/magicdog/workspace/magicdog-y1_sdk/build

# 语音控制示例
```

(下页继续)

(续上页)

```
./audio_example

# 传感器控制示例
./sensor_example

# 状态监控示例
./monitor_example

# 底层运动控制示例
./low_level_motion_example

# 高层运动控制示例
./high_level_motion_example
```

5.5 Python 例程示例

magicdog-y1_sdk/example/python 目录中：

- 语音示例:
 - audio_example.py
- 传感器示例:
 - sensor_example.py
- 状态监控示例:
 - monitor_example.py
- 底层运控示例:
 - low_level_motion_example.py
- 高层运控示例:
 - high_level_motion_example.py
- 最小建图导航示例:
 - minimal_slam_nav_example.py

Notice: 其中，高层运控示例中几种示例有先后依赖顺序（锁定站立-> 平衡站立-> 执行特技/遥控器控制）

5.5.1 进入调试模式：

按照操作流程，确保机器人进入调试模式

5.5.2 运行例程

进入 magicdog-y1_sdk/example/python 目录，执行如下命令：

```
cd /home/magicdog/workspace/magicdog-y1_sdk/example/python

# 语音控制示例
python3 audio_example.py

# 传感器控制示例
python3 sensor_example.py

# 状态监控示例
python3 monitor_example.py

# 底层运动控制示例
python3 low_level_motion_example.py

# 高层运动控制示例
python3 high_level_motion_example.py

# 最小建图导航示例
python3 minimal_slam_nav_example.py --local-ip 192.168.54.111
```


提供机器人系统主控制器，通过 **MagicRobot** 可以进行资源初始化、管理通信连接、访问各子控制器如运动控制器、语音、状态监控以及传感器控制器等。

6.1 接口定义

MagicRobot 是机器人 SDK 的统一入口类。

6.1.1 MagicRobot

项目	内容
函数名	MagicRobot
函数声明	MagicRobot();
功能概述	构造函数，创建 MagicRobot 实例。
备注	构造内部状态。

6.1.2 ~MagicRobot

项目	内容
函数名	<code>~MagicRobot</code>
函数声明	<code>~MagicRobot();</code>
功能概述	析构函数，释放 MagicRobot 实例资源。
备注	确保资源安全释放。

6.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize(const std::string& local_ip);</code>
功能概述	初始化机器人系统，包括控制器与网络模块。
参数说明	<code>local_ip</code> : 本地通信 IP 地址。用于后续通信身份绑定。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

6.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>void Shutdown();</code>
功能概述	关闭机器人系统，释放资源。
备注	配合 Initialize 使用。

6.1.5 Connect

项目	内容
函数名	<code>Connect</code>
函数声明	<code>Status Connect();</code>
功能概述	与机器人服务建立通信连接。
返回值	<code>Status::OK</code> 表示成功。
备注	阻塞接口，需初始化后调用。

6.1.6 Disconnect

项目	内容
函数名	Disconnect
函数声明	Status Disconnect();
功能概述	断开与机器人服务的连接。
返回值	gRPC 调用状态。
备注	阻塞接口，与 Connect 配对使用。

6.1.7 GetSDKVersion

项目	内容
函数名	GetSDKVersion
函数声明	std::string GetSDKVersion() const;
功能概述	获取当前 SDK 的版本号。
返回值	SDK 版本字符串（如 0.0.1）。
备注	非阻塞接口，便于调试或日志标记。

6.1.8 GetMotionControlLevel

项目	内容
函数名	GetMotionControlLevel
函数声明	ControllerLevel GetMotionControlLevel();
功能概述	获取当前运动控制级别（高层/低层）。
返回值	ControllerLevel 枚举值。
备注	非阻塞接口，用于判断当前控制权限。

6.1.9 SetMotionControlLevel

项目	内容
函数名	SetMotionControlLevel
函数声明	Status SetMotionControlLevel(ControllerLevel level);
功能概述	设置当前运动控制级别。
参数说明	level: 控制权限枚举值。
返回值	Status::OK 表示设置成功，其他代表设置失败。
备注	阻塞接口，控制模式切换时使用。

6.1.10 GetHighLevelMotionController

项目	内容
函数名	GetHighLevelMotionController
函数声明	HighLevelMotionController& GetHighLevelMotionController();
功能概述	获取高层运动控制器对象。
返回值	引用类型，用于调用高层控制接口。
备注	非阻塞接口，封装了步态、特技、遥控等控制。

6.1.11 GetLowLevelMotionController

项目	内容
函数名	GetLowLevelMotionController
函数声明	LowLevelMotionController& GetLowLevelMotionController();
功能概述	获取低层运动控制器对象。
返回值	引用类型，用于控制关节或电机。
备注	非阻塞接口，直接控制关节电机、获取 imu 等。

6.1.12 GetAudioController

项目	内容
函数名	GetAudioController
函数声明	AudioController& GetAudioController();
功能概述	获取音频控制器对象。
返回值	引用类型，用于语音控制。
备注	非阻塞接口，播放语音和控制音量。

6.1.13 GetSensorController

项目	内容
函数名	GetSensorController
函数声明	SensorController& GetSensorController();
功能概述	获取传感器控制器对象。
返回值	引用类型，用于访问传感器数据。
备注	非阻塞接口，封装了 IMU、RGBD 等读取。

6.1.14 GetStateMonitor

项目	内容
函数名	GetStateMonitor
函数声明	StateMonitor& GetStateMonitor();
功能概述	获取状态监控器对象。
返回值	引用类型，用于获取机器人当前状态信息。
备注	非阻塞接口，封装了 BMS、故障等状态信息的读取。

6.1.15 GetSlamNavController

项目	内容
函数名	GetSlamNavController
函数声明	SlamNavController& GetSlamNavController();
功能概述	获取 Slam Navigation 控制器对象。
返回值	引用类型，用于 Slam Navigation 控制。
备注	非阻塞接口，封装了建图、定位、导航等控制。

高层运动控制服务

提供机器人系统高层运动控制服务，通过 HighLevelMotionController 可以通过 RPC 通信方式实现对机器人的状态、运动模式、遥控的控制。

7.1 接口定义

HighLevelMotionController 是面向语义控制的高层运动控制器，支持如状态切换、运动模式、遥控等控制操作，封装底层细节以供上层系统调用。

7.1.1 HighLevelMotionController

项目	内容
函数名	HighLevelMotionController
函数声明	HighLevelMotionController();
功能概述	构造函数，初始化高层控制器状态
备注	构造内部控制资源

7.1.2 ~HighLevelMotionController

项目	内容
函数名	~HighLevelMotionController
函数声明	virtual ~HighLevelMotionController();
功能概述	析构函数，释放控制器资源
备注	配合构造函数使用

7.1.3 Initialize

项目	内容
函数名	Initialize
函数声明	virtual bool Initialize() override;
功能概述	初始化控制器，准备高层控制功能
返回值	true 表示成功，false 表示失败
备注	首次使用前必须调用

7.1.4 Shutdown

项目	内容
函数名	Shutdown
函数声明	virtual void Shutdown() override;
功能概述	关闭控制器并释放资源
备注	配合 Initialize 使用，安全断开连接

7.1.5 SetWalkGaitId

项目	内容
函数名	SetWalkGaitId
函数声明	Status SetWalkGaitId(const WalkGaitId walk_gait_id, int timeout_ms = 10000);
功能概述	设置行走步态 ID
参数说明	walk_gait_id: 行走步态枚举 timeout_ms: 接口调用的超时时间（单位：毫秒）。
返回值	Status::OK 表示成功，其他为失败
备注	阻塞接口，可切换多种步态模式

7.1.6 GetWalkGaitId

项目	内容
函数名	GetWalkGaitId
函数声明	Status GetWalkGaitId(WalkGaitId& walk_gait_id, int timeout_ms = 10000);
功能概述	获取当前行走步态 ID
参数说明	walk_gait_id: 行走步态枚举
返回值	Status::OK 表示成功，其他为失败
备注	阻塞接口，获取当前步态模式

7.1.7 SetMotionId

项目	内容
函数名	SetMotionId
函数声明	Status SetMotionId(MotionId motion_id, int timeout_ms = 10000);
功能概述	设置动作 ID
参数说明	motion_id: 动作枚举 timeout_ms: 接口调用的超时时间（单位：毫秒）。
返回值	Status::OK 表示成功，其他为失败
备注	阻塞接口，可切换多种动作

7.1.8 GetMotionId

项目	内容
函数名	GetMotionId
函数声明	Status GetMotionId(MotionId& motion_id, int timeout_ms = 10000);
功能概述	获取当前动作 ID
参数说明	motion_id: 动作枚举 timeout_ms: 接口调用的超时时间（单位：毫秒）。
返回值	Status::OK 表示成功，其他为失败
备注	阻塞接口。如果当前步态不是 MotionId 对应的动作步态，而是行走步态等其他类型，接口返回失败。

7.1.9 SendJoyStickCommand

项目	内容
函数名	SendJoyStickCommand
函数声明	Status SendJoyStickCommand(const JoystickCommand& joy_command);
功能概述	发送实时摇杆控制指令
参数说明	joy_command: 包含摇杆坐标的控制数据
返回值	Status::OK 表示成功, 其他为失败
备注	非阻塞接口, 建议发送频率为 20Hz

7.2 类型定义

7.2.1 JoystickCommand — 高层运动控制摇杆指令结构体

字段名	类型	描述
left_x_axis	float	左侧摇杆的 X 轴方向值 (-1.0: 左, 1.0: 右)
left_y_axis	float	左侧摇杆的 Y 轴方向值 (-1.0: 下, 1.0: 上)
right_x_axis	float	右侧摇杆的 X 轴方向值 (旋转 -1.0: 左, 1.0: 右)
right_y_axis	float	右侧摇杆的 Y 轴方向值 (暂未定义用途)

7.3 枚举类型定义

7.3.1 WalkGaitId — 机器人行走步态枚举

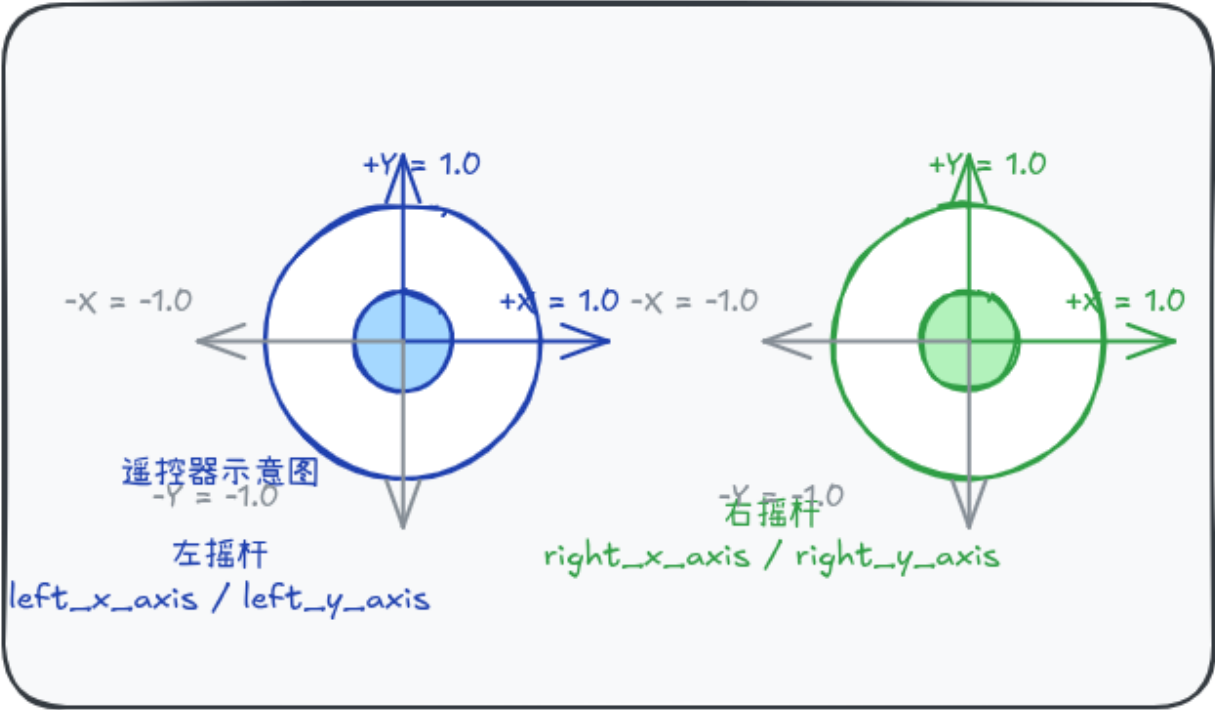
枚举值	数值	描述
WALK_GAIT_UNSPECIFIED	0	未指定
WALK_GAIT_TERRAIN	9	全地形行走
WALK_GAIT_LOAD	15	负载行走

7.3.2 MotionId — 机器人动作模式枚举

枚举值	数值	描述
MOTION_PASSIVE	0	空闲模式
MOTION_RECOVER_STAND	1	站立恢复
MOTION_CROUCH	2	蹲姿
MOTION_CHARGING_HALF_CROUCH	7	充电半蹲

7.4 遥控器示意图

遥控器示意图



取值范围： $[-1.0, 1.0]$ ；上 / 右为正方向

- 1. 左右摇杆 x 轴和 y 轴的取值范围为 $[-1.0, 1.0]$;
- 2. 左右摇杆 x 轴和 y 轴的方向上/右为正，如示意图所示；

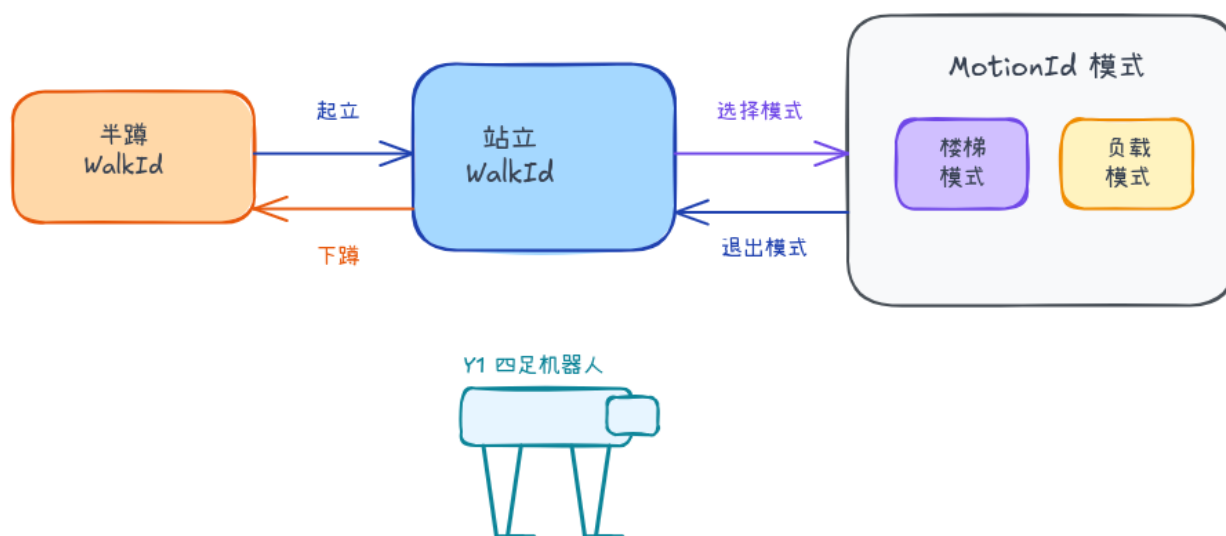
7.5 高层运动控制机器人状态介绍

机器人的运动包含站立、半蹲、楼梯模式、负载模式，机器人在运行过程中，通过状态机在不同状态之间进行切换，以实现不同的控制任务。各个状态的解释说明如下：

- **站立**：站立是机器人进入运动模式前的稳定状态。
- **半蹲**：半蹲用于降低机身高度，作为待机或模式切换前后的安全姿态。
- **楼梯模式**：楼梯模式用于台阶地形下的运动控制。
- **负载模式**：负载模式用于携带负载时的稳定运动控制。

机器人状态切换机制；

Y1 高层运动状态切换机制



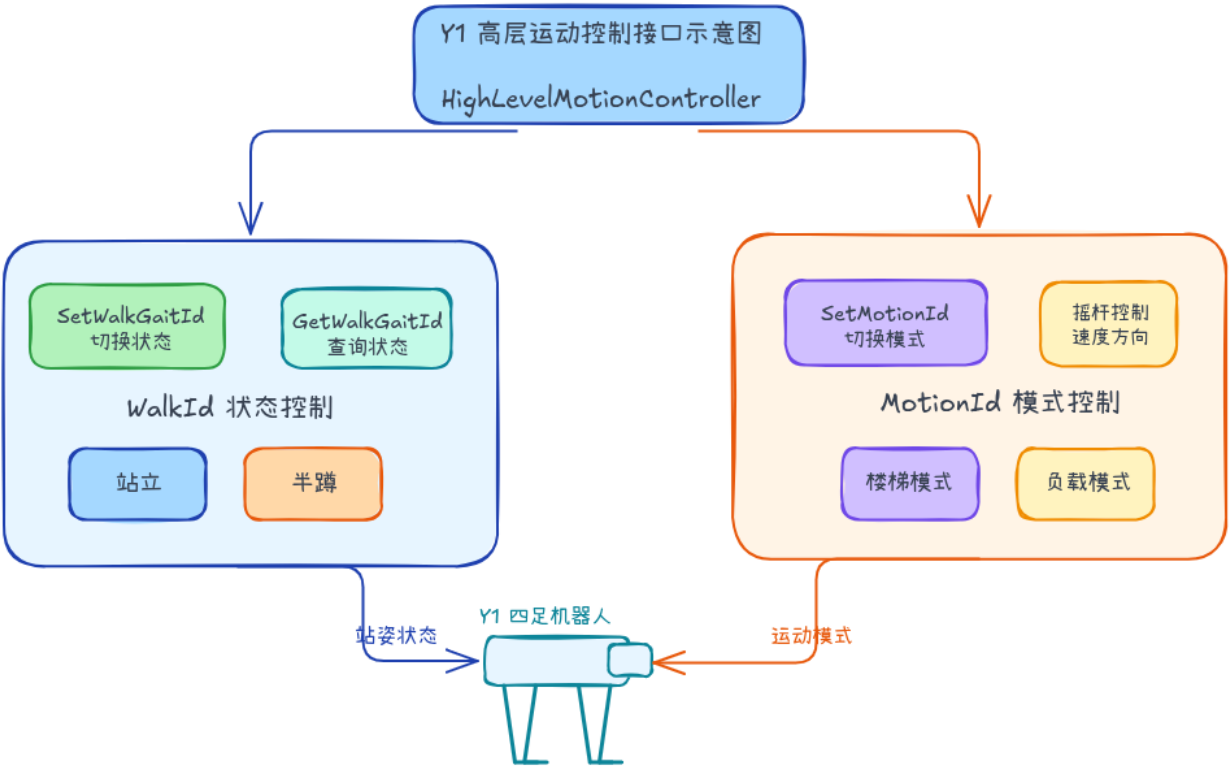
高层运动只表达 Y1 四足运动状态：WalkId 控制站立/半蹲，MotionId 控制楼梯/负载模式

7.6 高层运动控制接口

机器人的高层运动控制服务可分为 WalkId 状态控制、MotionId 模式控制和遥控控制。

- WalkId 状态控制中，可调用相应的接口切换站立、半蹲等基础状态。
- MotionId 模式控制中，可调用相应的接口切换楼梯模式、负载模式等运动模式。
- 遥控控制中，可发送摇杆指令控制机器人的速度和方向。

Y1 高层运动控制接口示意图



7.6.1 运动状态切换

控制目标	状态切换流程	示意图
站立		
半蹲		
楼梯模式		
负载模式		

CHAPTER 8

底层运动控制服务

该能力正在开发中，暂不开放公开文档。

请先使用高层运动控制接口完成站立、半蹲、楼梯模式、负载模式和遥控控制。

CHAPTER 9

传感器服务

该能力正在开发中，暂不开放公开文档。

后续版本会补充传感器数据接口、数据字段和调用示例。

CHAPTER 10

音频服务

该能力正在开发中，暂不开放公开文档。

后续版本会补充音频接口、参数说明和调用示例。

提供机器人状态监控接口类，通过 `StateMonitor` 可以提供状态查询等接口。

11.1 接口定义

`StateMonitor` 是机器人系统状态监控管理类，该类通常用于控制机器人的状态管理，支持状态查询。

11.1.1 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize();</code>
功能概述	初始化状态监控
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败
备注	首次使用前必须调用

11.1.2 Shutdown

项目	内容
函数名	Shutdown
函数声明	void Shutdown();
功能概述	关闭状态监控并释放资源
备注	配合 Initialize 使用，安全断开连接

11.1.3 GetCurrentState

项目	内容
函数名	GetCurrentState
函数声明	RobotState GetCurrentState();
功能概述	获取当前机器人聚合状态
返回值	RobotState 对象
备注	非阻塞接口，获取最近监控到的机器人状态数据，包括 BMS/故障状态监控等，后续会迭代拓展

11.1.4 RobotState —机器人状态数据结构体

用于表示机器人总体状态信息：

字段名	类型	描述
faults	vector<Fault>	故障信息列表
bms_data	BmsData	电池管理系统数据

11.1.5 BmsData —电池管理系统数据结构体

表示电池的状态信息：

字段名	类型	描述
battery_percentage	float	当前电池剩余电量（百分比，0~100）
battery_health	float	电池健康状态（越高越好）
battery_state	BatteryState	电池状态（见上方枚举）
power_supply_status	PowerSupplyStatus	电池充放电状态（见上方枚举）

11.1.6 枚举类型定义

BatteryState — 电池状态枚举类型

用于表示电池当前的状态，用于系统中电池状态的判断和处理：

枚举值	数值	描述
BatteryState::UNKNOWN	0	未知状态
BatteryState::GOOD	1	电池状态良好
BatteryState::OVERHEAT	2	电池过热
BatteryState::DEAD	3	电池损坏
BatteryState::OVERVOLTAGE	4	电池过电压
BatteryState::UNSPEC_FAILURE	5	未知故障
BatteryState::COLD	6	电池过冷
BatteryState::WATCHDOG_TIMER_EXPIRE	7	看门狗定时器超时
BatteryState::SAFETY_TIMER_EXPIRE	8	安全定时器超时

PowerSupplyStatus — 电池充放电状态

用于表示当前电池的充放电状态：

枚举值	数值	描述
PowerSupplyStatus::UNKNOWN	0	未知状态
PowerSupplyStatus::CHARGING	1	电池充电中
PowerSupplyStatus::DISCHARGING	2	电池放电中
PowerSupplyStatus::NOTCHARGING	3	电池未充放电
PowerSupplyStatus::FULL	4	电池充满

11.2 错误码映射表

错误码（十六进制）	错误描述
0x0000	No fault
0x1101	Service invocation failed
0x1301	Central control node lost
0x1302	App node lost
0x1303	Audio node lost
0x1304	Stereo camera node lost
0x1305	LIDAR node lost

下页继续

表 1 - 续上页

错误码（十六进制）	错误描述
0x1306	SLAM node lost
0x1307	Navigation node lost
0x1308	AI node lost
0x1309	Head node lost
0x130A	Point cloud node lost
0x2201	No LIDAR data received
0x2202	No stereo camera data received
0x2203	Stereo camera data error
0x2204	Stereo camera initialization failed
0x220B	No odometry data received
0x220C	No IMU data received
0x2215	Depth camera not detected
0x3101	Failed to connect robot to app
0x3102	Heartbeat lost - assertion failed
0x4201	Failed to open head serial port
0x4202	No head data received
0x5201	No navigation TF data
0x5202	No navigation map data
0x5203	No navigation localization data
0x5204	No navigation LIDAR data
0x5205	No navigation depth camera data
0x5206	No navigation multi-line LIDAR data
0x5207	No navigation odometry data
0x6201	SLAM localization error
0x6102	No SLAM LIDAR data
0x6103	No SLAM odometry data
0x6104	SLAM map data error
0x7201	LCM connection timeout
0x8201	Left leg hardware error
0x8202	Right leg hardware error
0x8203	Left arm hardware error
0x8204	Right arm hardware error
0x8205	Waist hardware error
0x8206	Head hardware error
0x8207	Hand hardware error
0x8208	Gripper hardware error
0x8209	IMU hardware error
0x820A	Power system hardware error

下页继续

表 1 - 续上页

错误码（十六进制）	错误描述
0x820B	Leg force sensor hardware error
0x820C	Arm force sensor hardware error
0x9201	ECAT (EtherCAT) hardware error
0xA201	Motion posture error
0xA202	Foot position deviation during movement
0xA203	Joint velocity error during motion

SLAM 导航控制服务

提供机器人系统 SLAM（同时定位与地图构建）和导航控制服务，通过 `SlamNavController` 可以通过 RPC 与话题订阅方式实现建图、定位、导航、地图管理与里程计数据获取。

12.1 接口定义

`SlamNavController` 是面向 SLAM 和导航控制的控制器，支持建图、定位、导航、地图管理与里程计相关控制操作，封装底层细节以供上层系统调用。

12.1.1 SlamNavController

项目	内容
函数名	<code>SlamNavController</code>
函数声明	<code>SlamNavController();</code>
功能概述	构造函数，初始化 SLAM 导航控制器状态。
备注	构造内部控制资源。

12.1.2 ~SlamNavController

项目	内容
函数名	<code>~SlamNavController</code>
函数声明	<code>~SlamNavController();</code>
功能概述	析构函数，释放控制器资源。
备注	配合构造函数使用。

12.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize();</code>
功能概述	初始化 SLAM 和导航控制器，准备相关资源。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

12.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>void Shutdown();</code>
功能概述	关闭控制器并释放资源。
备注	配合 <code>Initialize</code> 使用，安全断开相关连接。

12.1.5 ActivateSlamMode

项目	内容
函数名	ActivateSlamMode
函数声明	Status ActivateSlamMode(SlamMode mode, std::string map_path = "", int timeout_ms = 5000);
功能概述	切换 SLAM 模式并进入建图或定位流程。
参数说明	mode: SLAM 模式枚举 map_path: 定位模式下复用地图的绝对路径, 可通过 GetMapPath 获取 timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。进入 LOCALIZATION 模式时通常需要传入有效地图路径。

12.1.6 StartMapping

项目	内容
函数名	StartMapping
函数声明	Status StartMapping(int timeout_ms = 5000);
功能概述	开始建图。
参数说明	timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。建议先调用 ActivateSlamMode (SlamMode::MAPPING)。

12.1.7 CancelMapping

项目	内容
函数名	CancelMapping
函数声明	Status CancelMapping(int timeout_ms = 5000);
功能概述	取消当前建图任务。
参数说明	timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。用于结束当前建图流程但不保存地图。

12.1.8 SaveMap

项目	内容
函数名	SaveMap
函数声明	Status SaveMap(const std::string& map_name, int timeout_ms = 5000);
功能概述	结束建图并保存地图。
参数说明	map_name: 地图名称 timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。通常在建图完成后调用。

12.1.9 LoadMap

项目	内容
函数名	LoadMap
函数声明	Status LoadMap(const std::string& map_name, int timeout_ms = 5000);
功能概述	加载地图并设置为当前地图。
参数说明	map_name: 地图名称 timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。常用于定位或导航前的地图准备。

12.1.10 DeleteMap

项目	内容
函数名	DeleteMap
函数声明	Status DeleteMap(const std::string& map_name, int timeout_ms = 5000);
功能概述	删除地图。
参数说明	map_name: 要删除的地图名称 timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。删除后地图无法恢复。

12.1.11 GetMapPath

项目	内容
函数名	GetMapPath
函数声明	<code>Status GetMapPath(const std::string& map_name, std::vector<std::string>& map_path, int timeout_ms = 5000);</code>
功能概述	获取指定地图的存储路径。
参数说明	map_name: 地图名称 map_path: 地图路径列表 (输出参数) timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。返回结果可能包含一个或多个可用地图目录。

12.1.12 GetAllMapInfo

项目	内容
函数名	GetAllMapInfo
函数声明	<code>Status GetAllMapInfo(AllMapInfo& all_map_info, int timeout_ms = 5000);</code>
功能概述	获取当前地图和全部地图信息。
参数说明	all_map_info: 所有地图信息 (输出参数) timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。可用于获取地图列表以及地图元数据。

12.1.13 InitPose

项目	内容
函数名	InitPose
函数声明	Status InitPose(Pose3DEuler& pose, int timeout_ms = 5000);
功能概述	初始化机器人位姿。
参数说明	pose: 位姿信息 timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。由于激光雷达相对机器人正前方存在 -1.57rad 安装偏置, 期望的 yaw 需要额外减去 1.57rad , 否则位姿初始化可能失败; 如果流程较慢, 建议显式传入更大的 timeout_ms。

12.1.14 GetCurrentLocalizationInfo

项目	内容
函数名	GetCurrentLocalizationInfo
函数声明	Status GetCurrentLocalizationInfo(LocalizationInfo& pose_info);
功能概述	获取当前定位状态和位姿信息。
参数说明	pose_info: 当前位置和姿态信息 (输出参数)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。内部使用 SDK 默认超时获取当前定位结果。

12.1.15 ActivateNavMode

项目	内容
函数名	ActivateNavMode
函数声明	Status ActivateNavMode(NavMode mode, std::string map_path = "", int timeout_ms = 5000);
功能概述	激活导航模式。
参数说明	mode: 导航模式枚举 map_path: 复用地图的绝对路径, 通常可通过 GetMapPath 获取 timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。进入 GRID_MAP 模式时通常需要先准备有效地图路径。

12.1.16 SetNavTarget

项目	内容
函数名	SetNavTarget
函数声明	Status SetNavTarget(const NavTarget& goal, int timeout_ms = 5000);
功能概述	设置导航目标点并启动导航任务。
参数说明	goal: 导航目标点 timeout_ms: 超时时间（毫秒）
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。目标点通常使用 map 坐标系。

12.1.17 PauseNavTask

项目	内容
函数名	PauseNavTask
函数声明	Status PauseNavTask();
功能概述	暂停当前导航任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。仅等待控制 RPC 返回，不等待导航状态完全稳定。

12.1.18 ResumeNavTask

项目	内容
函数名	ResumeNavTask
函数声明	Status ResumeNavTask();
功能概述	恢复暂停的导航任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。仅等待控制 RPC 返回。

12.1.19 CancelNavTask

项目	内容
函数名	CancelNavTask
函数声明	Status CancelNavTask();
功能概述	取消当前导航任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。仅等待取消请求返回。

12.1.20 GetNavTaskStatus

项目	内容
函数名	GetNavTaskStatus
函数声明	Status GetNavTaskStatus (NavStatus& nav_status);
功能概述	获取当前导航任务状态。
参数说明	nav_status: 导航状态（输出参数）
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。用于轮询导航执行进度、结束状态和错误信息。

12.1.21 OpenOdometryStream

项目	内容
函数名	OpenOdometryStream
函数声明	Status OpenOdometryStream();
功能概述	打开里程计数据流。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。通常与 SubscribeOdometry 配合使用。

12.1.22 CloseOdometryStream

项目	内容
函数名	CloseOdometryStream
函数声明	Status CloseOdometryStream();
功能概述	关闭里程计数据流。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。与 OpenOdometryStream 配合使用。

12.1.23 SubscribeOdometry

项目	内容
函数名	SubscribeOdometry
函数声明	<code>void SubscribeOdometry(const OdometryCallback callback);</code>
功能概述	订阅里程计数据。
参数说明	callback: 接收到里程计数据后的处理回调函数, 参数类型为 <code>std::shared_ptr<Odometry></code> ;
备注	非阻塞接口。需要先打开里程计数据流。

12.1.24 UnsubscribeOdometry

项目	内容
函数名	UnsubscribeOdometry
函数声明	<code>void UnsubscribeOdometry();</code>
功能概述	取消订阅里程计数据。
备注	非阻塞接口。与 SubscribeOdometry 配合使用。

12.1.25 GetPointCloudMap

项目	内容
函数名	GetPointCloudMap
函数声明	<code>Status GetPointCloudMap(PointCloud2& point_cloud_map, int timeout_ms = 5000);</code>
功能概述	获取当前点云地图数据。
参数说明	point_cloud_map: 点云地图数据 (输出参数) timeout_ms: 超时时间 (毫秒)
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。返回数据结构与传感器服务文档中的 PointCloud2 定义一致。

12.2 类型定义

12.2.1 SlamMode —SLAM 模式枚举

枚举值	数值	描述
IDLE	0	空闲模式
MAPPING	1	建图模式
LOCALIZATION	3	定位模式

12.2.2 NavMode —导航模式枚举

枚举值	数值	描述
IDLE	0	空闲模式
GRID_MAP	13	栅格地图导航模式

12.2.3 Pose3DEuler —3D 位姿结构体

字段名	类型	描述
position	std::array<double, 3>	位置坐标 (x, y, z)
orientation	std::array<double, 3>	欧拉角 (roll, pitch, yaw)

12.2.4 MapImageData —地图图像数据结构体

字段名	类型	描述
type	std::string	图像魔数, 例如 P5
width	uint32_t	图像宽度
height	uint32_t	图像高度
max_gray_value	uint32_t	最大灰度值
image	std::vector<uint8_t>	原始图像数据

12.2.5 MapMetaData —地图元数据结构体

字段名	类型	描述
resolution	double	地图分辨率, 单位 m/pixel
origin	Pose3DEuler	地图原点
map_image_data	MapImageData	地图图像数据

12.2.6 MapInfo —单张地图信息结构体

字段名	类型	描述
map_name	std::string	地图名称
map_meta_data	MapMetaData	地图元数据

12.2.7 AllMapInfo —地图集合结构体

字段名	类型	描述
current_map_name	std::string	当前地图名称
map_infos	std::vector<MapInfo>	全部地图信息列表

12.2.8 LocalizationInfo —定位信息结构体

字段名	类型	描述
is_localization	bool	是否已完成定位
pose	Pose3DEuler	当前位姿

12.2.9 NavTarget —导航目标点结构体

字段名	类型	描述
id	int32_t	目标点 ID
frame_id	std::string	目标点坐标系 ID
goal	Pose3DEuler	目标点位姿

12.2.10 NavStatusType — 导航状态枚举

枚举值	数值	描述
NONE	0	无状态
RUNNING	1	导航进行中
END_SUCCESS	2	导航成功结束
END_FAILED	3	导航失败结束
PAUSE	4	导航暂停
CONTINUE	5	导航继续
CANCEL	6	导航取消

12.2.11 NavStatus — 导航状态结构体

字段名	类型	描述
id	int32_t	目标点 ID, -1 表示当前没有目标点
status	NavStatusType	导航状态
error_code	int32_t	导航错误码
error_desc	std::string	导航错误描述

12.2.12 Odometry — 里程计数据结构体

字段名	类型	描述
header	Header	消息头
child_frame_id	std::string	子坐标系 ID
position	std::array<double, 3>	位置坐标 (x, y, z)
orientation	std::array<double, 4>	四元数姿态 (w, x, y, z)
linear_velocity	std::array<double, 3>	线速度 (x, y, z)
angular_velocity	std::array<double, 3>	角速度 (x, y, z)

机器人主控服务 (Python)

提供机器人系统主控制器，通过 `MagicRobot` 类可以进行资源初始化、管理通信连接、访问各子控制器如运动控制器、语音、状态监控以及传感器控制器等。

13.1 接口定义

`MagicRobot` 是机器人 SDK 的统一入口类。

13.1.1 MagicRobot

项目	内容
类名	<code>MagicRobot</code>
构造函数	<code>robot = MagicRobot()</code>
功能概述	构造函数，创建 <code>MagicRobot</code> 实例。
备注	构造内部状态。

13.1.2 initialize

项目	内容
函数名	<code>initialize</code>
函数声明	<code>bool initialize(str local_ip)</code>
功能概述	初始化机器人系统，包括控制器与网络模块。
参数说明	<code>local_ip</code> : 本地通信 IP 地址。
返回值	<code>True</code> 表示成功， <code>False</code> 表示失败。
备注	首次使用前必须调用。

13.1.3 shutdown

项目	内容
函数名	<code>shutdown</code>
函数声明	<code>void shutdown()</code>
功能概述	关闭机器人系统，释放资源。
备注	配合 <code>initialize</code> 使用。

13.1.4 connect

项目	内容
函数名	<code>connect</code>
函数声明	<code>Status connect()</code>
功能概述	与机器人服务建立通信连接。
返回值	<code>Status</code> 对象， <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口，需初始化后调用。

13.1.5 disconnect

项目	内容
函数名	<code>disconnect</code>
函数声明	<code>Status disconnect()</code>
功能概述	断开与机器人服务的连接。
返回值	<code>Status</code> 对象， <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口，与 <code>connect</code> 配对使用。

13.1.6 get_sdk_version

项目	内容
函数名	get_sdk_version
函数声明	str get_sdk_version()
功能概述	获取当前 SDK 的版本号。
返回值	SDK 版本字符串（如 0.0.1）。
备注	非阻塞接口，便于调试或日志标记。

13.1.7 get_motion_control_level

项目	内容
函数名	get_motion_control_level
函数声明	ControllerLevel get_motion_control_level()
功能概述	获取当前运动控制级别（高层/低层）。
返回值	ControllerLevel 枚举值。
备注	非阻塞接口，用于判断当前控制权限。

13.1.8 set_motion_control_level

项目	内容
函数名	set_motion_control_level
函数声明	Status set_motion_control_level(ControllerLevel level)
功能概述	设置当前运动控制级别。
参数说明	level: 控制权限枚举值。
返回值	Status 对象, Status.code == ErrorCode.OK 表示设置成功。
备注	阻塞接口，控制模式切换时使用。

13.1.9 get_high_level_motion_controller

项目	内容
函数名	get_high_level_motion_controller
函数声明	HighLevelMotionController get_high_level_motion_controller()
功能概述	获取高层运动控制器对象。
返回值	HighLevelMotionController 对象，用于调用高层控制接口。
备注	非阻塞接口，封装了步态、特技、遥控等控制。

13.1.10 get_low_level_motion_controller

项目	内容
函数名	get_low_level_motion_controller
函数声明	LowLevelMotionController get_low_level_motion_controller()
功能概述	获取低层运动控制器对象。
返回值	LowLevelMotionController 对象，用于控制关节或电机。
备注	非阻塞接口，直接控制关节电机、获取 imu 等。

13.1.11 get_audio_controller

项目	内容
函数名	get_audio_controller
函数声明	AudioController get_audio_controller()
功能概述	获取音频控制器对象。
返回值	AudioController 对象，用于语音控制。
备注	非阻塞接口，播放语音和控制音量。

13.1.12 get_sensor_controller

项目	内容
函数名	get_sensor_controller
函数声明	SensorController get_sensor_controller()
功能概述	获取传感器控制器对象。
返回值	SensorController 对象，用于访问传感器数据。
备注	非阻塞接口，封装了 IMU、RGBD 等读取。

13.1.13 get_state_monitor

项目	内容
函数名	get_state_monitor
函数声明	StateMonitor get_state_monitor()
功能概述	获取状态监控器对象。
返回值	StateMonitor 对象，用于获取机器人当前状态信息。
备注	非阻塞接口，封装了 BMS、故障等状态信息的读取。

13.1.14 get_slam_nav_controller

项目	内容
函数名	get_slam_nav_controller
函数声明	SlamNavController get_sensor_controller()
功能概述	获取 Slam Navigation 控制器对象。
返回值	SlamNavController 对象，用于 Slam Navigation 控制。
备注	非阻塞接口，封装了建图、定位、导航等控制。

高层运动控制服务 (Python)

提供机器人系统高层运动控制服务，通过 `HighLevelMotionController` 可以通过 RPC 通信方式实现对机器人的状态、运动模式、遥控的控制。

14.1 接口定义

`HighLevelMotionController` 是面向语义控制的高层运动控制器，支持如状态切换、运动模式、遥控等控制操作，封装底层细节以供上层系统调用。

14.1.1 `HighLevelMotionController`

项目	内容
类名	<code>HighLevelMotionController</code>
构造函数	<code>controller = HighLevelMotionController()</code>
功能概述	构造函数，初始化高层控制器状态
备注	构造内部控制资源

14.1.2 initialize

项目	内容
函数名	<code>initialize</code>
函数声明	<code>bool initialize()</code>
功能概述	初始化控制器，准备高层控制功能
返回值	<code>True</code> 表示成功， <code>False</code> 表示失败
备注	首次使用前必须调用

14.1.3 shutdown

项目	内容
函数名	<code>shutdown</code>
函数声明	<code>void shutdown()</code>
功能概述	关闭控制器并释放资源
备注	配合 <code>initialize</code> 使用，安全断开连接

14.1.4 set_walk_gait_id

项目	内容
函数名	<code>set_walk_gait_id</code>
函数声明	<code>Status set_walk_gait_id(WalkGaitId walk_gait_id)</code>
功能概述	设置行走步态 ID
参数说明	<code>walk_gait_id</code> : 行走步态枚举
返回值	<code>Status</code> 对象， <code>Status.code == ErrorCode.OK</code> 表示成功
备注	阻塞接口，可切换多种步态模式

14.1.5 get_walk_gait_id

项目	内容
函数名	<code>get_walk_gait_id</code>
函数声明	<code>Status get_walk_gait_id()</code>
功能概述	获取当前行走步态 ID
返回值	<code>Status</code> 对象，包含当前行走步态信息
备注	阻塞接口，获取当前行走步态

14.1.6 set_motion_id

项目	内容
函数名	set_motion_id
函数声明	Status set_motion_id(MotionId motion_id)
功能概述	设置非行走动作 ID
参数说明	motion_id: 动作枚举
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功

14.1.7 get_motion_id

项目	内容
函数名	get_motion_id
函数声明	Status get_motion_id()
功能概述	获取当前动作 ID
返回值	返回 (status, motion_id) 元组; status.code == ErrorCode.OK 表示成功
备注	阻塞接口。如果当前步态不是 MotionId 对应的动作步态, 而是行走步态等其他类型, 接口返回失败。

14.1.8 send_joystick_command

项目	内容
函数名	send_joystick_command
函数声明	Status send_joystick_command(JoystickCommand joy_command)
功能概述	发送实时摇杆控制指令
参数说明	joy_command: 包含摇杆坐标的控制数据
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功
备注	非阻塞接口, 建议发送频率为 20Hz

14.2 类型定义

14.2.1 JoystickCommand —高层运动控制摇杆指令结构体

字段名	类型	描述
left_x_axis	float	左侧摇杆的 X 轴方向值 (-1.0: 左, 1.0: 右)
left_y_axis	float	左侧摇杆的 Y 轴方向值 (-1.0: 下, 1.0: 上)
right_x_axis	float	右侧摇杆的 X 轴方向值 (旋转 -1.0: 左, 1.0: 右)
right_y_axis	float	右侧摇杆的 Y 轴方向值 (暂未定义用途)

14.3 枚举类型定义

14.3.1 WalkGaitId —机器人行走步态枚举

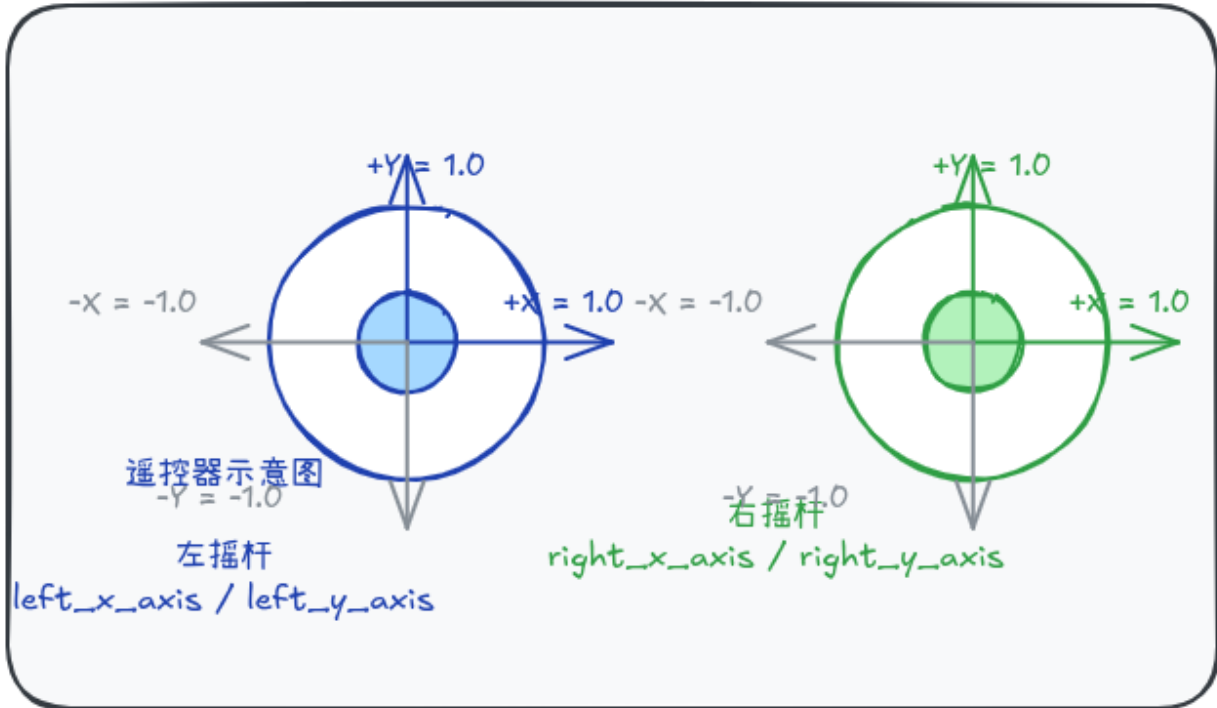
枚举值	数值	描述
WALK_GAIT_UNSPECIFIED	0	未指定
WALK_GAIT_TERRAIN	9	全地形行走
WALK_GAIT_LOAD	15	负载行走

14.3.2 MotionId —机器人动作模式枚举

枚举值	数值	描述
MOTION_PASSIVE	0	空闲模式
MOTION_RECOVER_STAND	1	站立恢复
MOTION_CROUCH	2	蹲姿
MOTION_CHARGING_HALF_CROUCH	7	充电半蹲

14.4 遥控器示意图

遥控器示意图



取值范围：[-1.0, 1.0]；上 / 右为正方向

1. 左右摇杆 x 轴和 y 轴的取值范围为 [-1.0, 1.0];
2. 左右摇杆 x 轴和 y 轴的方向上/右为正，如示意图所示；

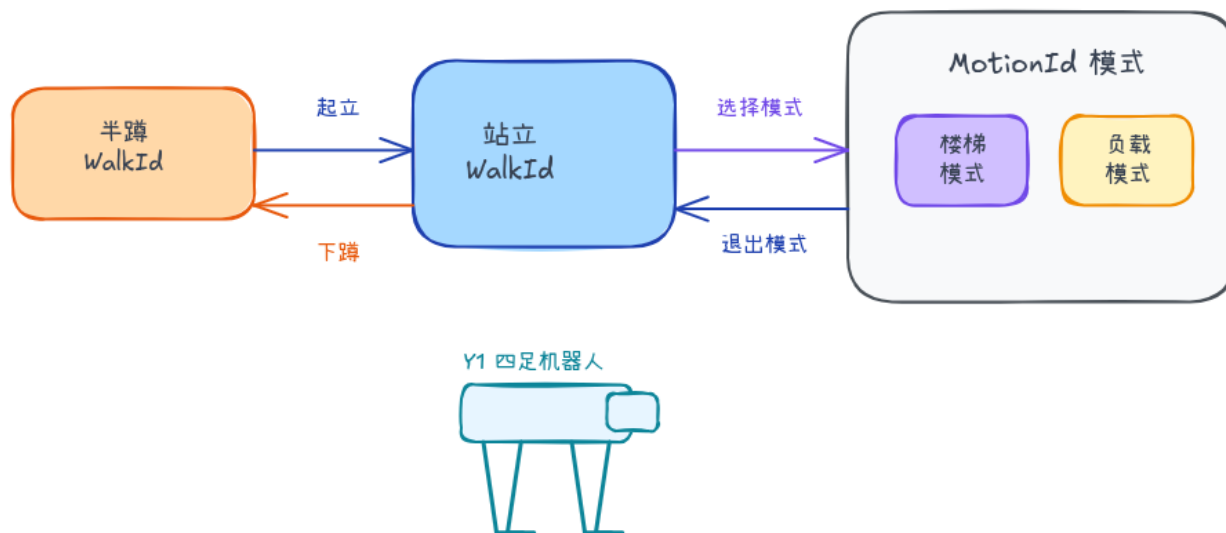
14.5 高层运动控制机器人状态介绍

机器人的运动包含站立、半蹲、楼梯模式、负载模式，机器人在运行过程中，通过状态机在不同状态之间进行切换，以实现不同的控制任务。各个状态的解释说明如下：

- **站立**：站立是机器人进入运动模式前的稳定状态。
- **半蹲**：半蹲用于降低机身高度，作为待机或模式切换前后的安全姿态。
- **楼梯模式**：楼梯模式用于台阶地形下的运动控制。
- **负载模式**：负载模式用于携带负载时的稳定运动控制。

机器人状态切换机制；

Y1 高层运动状态切换机制



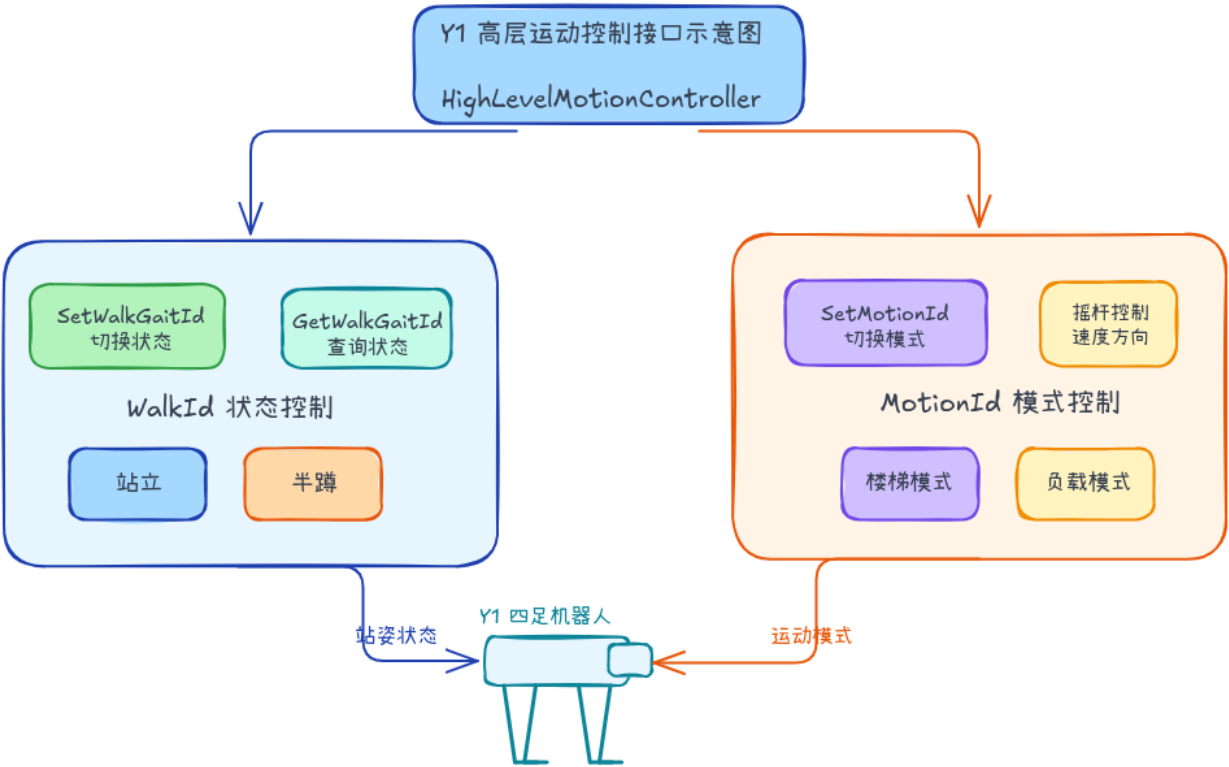
高层运动只表达 Y1 四足运动状态：WalkId 控制站立/半蹲，MotionId 控制楼梯/负载模式

14.6 高层运动控制接口

机器人的高层运动控制服务可分为 WalkId 状态控制、MotionId 模式控制和遥控控制。

- WalkId 状态控制中，可调用相应的接口切换站立、半蹲等基础状态。
- MotionId 模式控制中，可调用相应的接口切换楼梯模式、负载模式等运动模式。
- 遥控控制中，可发送摇杆指令控制机器人的速度和方向。

Y1 高层运动控制接口示意图



14.6.1 运动状态切换

控制目标	状态切换流程	示意图
站立		
半蹲		
楼梯模式		
负载模式		

CHAPTER 15

底层运动控制服务

该能力正在开发中，暂不开放公开文档。

请先使用高层运动控制接口完成站立、半蹲、楼梯模式、负载模式和遥控控制。

CHAPTER 16

传感器服务

该能力正在开发中，暂不开放公开文档。

后续版本会补充传感器数据接口、数据字段和调用示例。

CHAPTER 17

音频服务

该能力正在开发中，暂不开放公开文档。

后续版本会补充音频接口、参数说明和调用示例。

状态监控服务 (Python)

提供机器人状态监控接口类，通过 `StateMonitor` 可以提供状态查询等接口。

18.1 接口定义

`StateMonitor` 是机器人系统状态监控管理类，该类通常用于控制机器人的状态管理，支持状态查询。

18.1.1 initialize

项目	内容
函数名	<code>initialize</code>
函数声明	<code>bool initialize()</code>
功能概述	初始化状态监控。
返回值	<code>True</code> 表示成功， <code>False</code> 表示失败。
备注	首次调用必须初始化。

18.1.2 shutdown

项目	内容
函数名	shutdown
函数声明	void shutdown()
功能概述	关闭状态监控，释放资源。
备注	配合 initialize 使用。

18.1.3 get_current_state

项目	内容
函数名	get_current_state
函数声明	RobotState get_current_state()
功能概述	获取当前机器人聚合状态
返回值	RobotState 对象
备注	非阻塞接口，获取最近监控到的机器人状态数据，包括 BMS/故障状态监控等，后续会迭代拓展

18.1.4 RobotState — 机器人状态数据结构体

用于表示机器人总体状态信息：

字段名	类型	描述
faults	list[Fault]	故障信息列表
bms_data	BmsData	电池管理系统数据

18.1.5 BmsData — 电池管理系统数据结构体

表示电池的状态信息：

字段名	类型	描述
battery_percentage	float	当前电池剩余电量（百分比，0~100）
battery_health	float	电池健康状态（越高越好）
battery_state	BatteryState	电池状态（见下方枚举）
power_supply_status	PowerSupplyStatus	电池充放电状态（见下方枚举）

18.1.6 枚举类型定义

BatteryState — 电池状态枚举类型

用于表示电池当前的状态，用于系统中电池状态的判断和处理：

枚举值	数值	描述
BatteryState.UNKNOWN	0	未知状态
BatteryState.GOOD	1	电池状态良好
BatteryState.OVERHEAT	2	电池过热
BatteryState.DEAD	3	电池损坏
BatteryState.OVERVOLTAGE	4	电池过电压
BatteryState.UNSPEC_FAILURE	5	未知故障
BatteryState.COLD	6	电池过冷
BatteryState.WATCHDOG_TIMER_EXPIRE	7	看门狗定时器超时
BatteryState.SAFETY_TIMER_EXPIRE	8	安全定时器超时

PowerSupplyStatus — 电池充放电状态

用于表示当前电池的充放电状态：

枚举值	数值	描述
PowerSupplyStatus.UNKNOWN	0	未知状态
PowerSupplyStatus.CHARGING	1	电池充电中
PowerSupplyStatus.DISCHARGING	2	电池放电中
PowerSupplyStatus.NOTCHARGING	3	电池未充放电
PowerSupplyStatus.FULL	4	电池充满

18.2 错误码映射表

错误码（十六进制）	错误描述
0x0000	No fault
0x1101	Service invocation failed
0x1301	Central control node lost
0x1302	App node lost
0x1303	Audio node lost
0x1304	Stereo camera node lost
0x1305	LIDAR node lost

下页继续

表 1 - 续上页

错误码（十六进制）	错误描述
0x1306	SLAM node lost
0x1307	Navigation node lost
0x1308	AI node lost
0x1309	Head node lost
0x130A	Point cloud node lost
0x2201	No LIDAR data received
0x2202	No stereo camera data received
0x2203	Stereo camera data error
0x2204	Stereo camera initialization failed
0x220B	No odometry data received
0x220C	No IMU data received
0x2215	Depth camera not detected
0x3101	Failed to connect robot to app
0x3102	Heartbeat lost - assertion failed
0x4201	Failed to open head serial port
0x4202	No head data received
0x5201	No navigation TF data
0x5202	No navigation map data
0x5203	No navigation localization data
0x5204	No navigation LIDAR data
0x5205	No navigation depth camera data
0x5206	No navigation multi-line LIDAR data
0x5207	No navigation odometry data
0x6201	SLAM localization error
0x6102	No SLAM LIDAR data
0x6103	No SLAM odometry data
0x6104	SLAM map data error
0x7201	LCM connection timeout
0x8201	Left leg hardware error
0x8202	Right leg hardware error
0x8203	Left arm hardware error
0x8204	Right arm hardware error
0x8205	Waist hardware error
0x8206	Head hardware error
0x8207	Hand hardware error
0x8208	Gripper hardware error
0x8209	IMU hardware error
0x820A	Power system hardware error

下页继续

表 1 - 续上页

错误码（十六进制）	错误描述
0x820B	Leg force sensor hardware error
0x820C	Arm force sensor hardware error
0x9201	ECAT (EtherCAT) hardware error
0xA201	Motion posture error
0xA202	Foot position deviation during movement
0xA203	Joint velocity error during motion

SLAM 导航控制服务 (Python)

提供机器人系统 SLAM（同时定位与地图构建）和导航控制服务，通过 `SlamNavController` 可以通过 RPC 与话题订阅方式实现建图、定位、导航、地图管理与里程计数据获取。

19.1 接口定义

`SlamNavController` 是面向 SLAM 和导航控制的控制器，支持建图、定位、导航、地图管理与里程计相关控制操作，封装底层细节以供上层系统调用。

19.1.1 SlamNavController

项目	内容
类名	<code>SlamNavController</code>
类声明	<code>controller = SlamNavController()</code>
功能概述	构造函数，初始化 SLAM 导航控制器状态。
备注	构造内部控制资源。

19.1.2 initialize

项目	内容
方法名	<code>initialize</code>
方法声明	<code>bool initialize()</code>
功能概述	初始化 SLAM 和导航控制器，准备相关资源。
返回值	<code>True</code> 表示成功， <code>False</code> 表示失败。
备注	首次使用前必须调用。

19.1.3 shutdown

项目	内容
方法名	<code>shutdown</code>
方法声明	<code>void shutdown()</code>
功能概述	关闭控制器并释放资源。
备注	配合 <code>initialize</code> 使用，安全断开相关连接。

19.1.4 activate_slam_mode

项目	内容
方法名	<code>activate_slam_mode</code>
方法声明	<code>Status activate_slam_mode(SlamMode mode, str map_path = "", int timeout_ms = 10000)</code>
功能概述	切换 SLAM 模式并进入建图或定位流程。
参数说明	mode: SLAM 模式枚举 map_path: 定位模式下复用地图的绝对路径，可通过 <code>get_map_path</code> 获取 timeout_ms: 超时时间（毫秒）
返回值	<code>Status</code> 对象， <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。进入 <code>LOCALIZATION</code> 模式时通常需要传入有效地图路径。

19.1.5 start_mapping

项目	内容
方法名	start_mapping
方法声明	Status start_mapping(int timeout_ms = 10000)
功能概述	开始建图。
参数说明	timeout_ms: 超时时间（毫秒）
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。建议先调用 activate_slam_mode(SlamMode.MAPPING)。

19.1.6 cancel_mapping

项目	内容
方法名	cancel_mapping
方法声明	Status cancel_mapping(int timeout_ms = 10000)
功能概述	取消当前建图任务。
参数说明	timeout_ms: 超时时间（毫秒）
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。用于结束当前建图流程但不保存地图。

19.1.7 save_map

项目	内容
方法名	save_map
方法声明	Status save_map(str map_name, int timeout_ms = 10000)
功能概述	结束建图并保存地图。
参数说明	map_name: 地图名称 timeout_ms: 超时时间（毫秒）
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。通常在建图完成后调用。

19.1.8 load_map

项目	内容
方法名	load_map
方法声明	Status load_map(str map_name, int timeout_ms = 10000)
功能概述	加载地图并设置为当前地图。
参数说明	map_name: 地图名称 timeout_ms: 超时时间 (毫秒)
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。常用于定位或导航前的地图准备。

19.1.9 delete_map

项目	内容
方法名	delete_map
方法声明	Status delete_map(str map_name, int timeout_ms = 10000)
功能概述	删除地图。
参数说明	map_name: 要删除的地图名称 timeout_ms: 超时时间 (毫秒)
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。删除后地图无法恢复。

19.1.10 get_map_path

项目	内容
方法名	get_map_path
方法声明	tuple(Status, list[str]) get_map_path(str map_name, int timeout_ms = 10000)
功能概述	获取指定地图的存储路径。
参数说明	map_name: 地图名称 timeout_ms: 超时时间 (毫秒)
返回值	返回 (status, map_path) 元组; status.code == ErrorCode.OK 表示成功, map_path 为地图路径列表。
备注	阻塞接口。返回结果可能包含一个或多个可用地图目录。

19.1.11 get_all_map_info

项目	内容
方法名	<code>get_all_map_info</code>
方法声明	<code>tuple[Status, AllMapInfo] get_all_map_info(int timeout_ms = 10000)</code>
功能概述	获取当前地图和全部地图信息。
参数说明	<code>timeout_ms</code> : 超时时间（毫秒）
返回值	返回 <code>(status, all_map_info)</code> 元组； <code>status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。可用于获取地图列表以及地图元数据。

19.1.12 init_pose

项目	内容
方法名	<code>init_pose</code>
方法声明	<code>Status init_pose(Pose3DEuler pose, int timeout_ms = 10000)</code>
功能概述	初始化机器人位姿。
参数说明	<code>pose</code> : 位姿信息 <code>timeout_ms</code> : 超时时间（毫秒）
返回值	<code>Status</code> 对象， <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。由于激光雷达相对机器人正前方存在 -1.57rad 安装偏置，期望的 <code>yaw</code> 需要额外减去 1.57rad ，否则位姿初始化可能失败；如流程较慢，可显式传入更大的 <code>timeout_ms</code> 。

19.1.13 get_current_localization_info

项目	内容
方法名	<code>get_current_localization_info</code>
方法声明	<code>tuple[Status, LocalizationInfo] get_current_localization_info()</code>
功能概述	获取当前定位状态和位姿信息。
返回值	返回 <code>(status, pose_info)</code> 元组； <code>status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。内部使用 SDK 默认超时获取当前定位结果。

19.1.14 activate_nav_mode

项目	内容
方法名	activate_nav_mode
方法声明	Status activate_nav_mode(NavMode mode, str map_path = "", int timeout_ms = 10000)
功能概述	激活导航模式。
参数说明	mode: 导航模式枚举 map_path: 复用地图的绝对路径, 通常可通过 get_map_path 获取 timeout_ms: 超时时间 (毫秒)
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。进入 GRID_MAP 模式时通常需要先准备有效地图路径。

19.1.15 set_nav_target

项目	内容
方法名	set_nav_target
方法声明	Status set_nav_target(NavTarget goal, int timeout_ms = 10000)
功能概述	设置导航目标点并启动导航任务。
参数说明	goal: 导航目标点 timeout_ms: 超时时间 (毫秒)
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。目标点通常使用 map 坐标系。

19.1.16 pause_nav_task

项目	内容
方法名	pause_nav_task
方法声明	Status pause_nav_task()
功能概述	暂停当前导航任务。
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。仅等待控制 RPC 返回, 不等待导航状态完全稳定。

19.1.17 resume_nav_task

项目	内容
方法名	<code>resume_nav_task</code>
方法声明	<code>Status resume_nav_task()</code>
功能概述	恢复暂停的导航任务。
返回值	<code>Status</code> 对象, <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。仅等待控制 RPC 返回。

19.1.18 cancel_nav_task

项目	内容
方法名	<code>cancel_nav_task</code>
方法声明	<code>Status cancel_nav_task()</code>
功能概述	取消当前导航任务。
返回值	<code>Status</code> 对象, <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。仅等待取消请求返回。

19.1.19 get_nav_task_status

项目	内容
方法名	<code>get_nav_task_status</code>
方法声明	<code>tuple[Status, NavStatus] get_nav_task_status()</code>
功能概述	获取当前导航任务状态。
返回值	返回 <code>(status, nav_status)</code> 元组; <code>status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。用于轮询导航执行进度、结束状态和错误信息。

19.1.20 open_odometry_stream

项目	内容
方法名	<code>open_odometry_stream</code>
方法声明	<code>Status open_odometry_stream()</code>
功能概述	打开里程计数据流。
返回值	<code>Status</code> 对象, <code>Status.code == ErrorCode.OK</code> 表示成功。
备注	阻塞接口。通常与 <code>subscribe_odometry</code> 配合使用。

19.1.21 close_odometry_stream

项目	内容
方法名	close_odometry_stream
方法声明	Status close_odometry_stream()
功能概述	关闭里程计数据流。
返回值	Status 对象, Status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。与 open_odometry_stream 配合使用。

19.1.22 subscribe_odometry

项目	内容
方法名	subscribe_odometry
方法声明	void subscribe_odometry(callback)
功能概述	订阅里程计数据。
参数说明	callback: 接收到里程计数据后的处理函数, 签名为 callback(data: Odometry) -> None
备注	非阻塞接口。需要先打开里程计数据流。

19.1.23 unsubscribe_odometry

项目	内容
方法名	unsubscribe_odometry
方法声明	void unsubscribe_odometry()
功能概述	取消订阅里程计数据。
备注	非阻塞接口。与 subscribe_odometry 配合使用。

19.1.24 get_point_cloud_map

项目	内容
方法名	get_point_cloud_map
方法声明	tuple[Status, PointCloud2] get_point_cloud_map(int timeout_ms = 10000)
功能概述	获取当前点云地图数据。
参数说明	timeout_ms: 超时时间 (毫秒)
返回值	返回 (status, point_cloud_map) 元组; status.code == ErrorCode.OK 表示成功。
备注	阻塞接口。返回数据结构与传感器服务文档中的 PointCloud2 定义一致。

19.2 类型定义

19.2.1 SlamMode —SLAM 模式枚举

枚举值	数值	描述
IDLE	0	空闲模式
MAPPING	1	建图模式
LOCALIZATION	3	定位模式

19.2.2 NavMode —导航模式枚举

枚举值	数值	描述
IDLE	0	空闲模式
GRID_MAP	13	栅格地图导航模式

19.2.3 Pose3DEuler —3D 位姿结构体

字段名	类型	描述
position	list[float]	位置坐标 [x, y, z]
orientation	list[float]	欧拉角 [roll, pitch, yaw]

19.2.4 MapImageData —地图图像数据结构体

字段名	类型	描述
type	str	图像魔数, 例如 P5
width	int	图像宽度
height	int	图像高度
max_gray_value	int	最大灰度值
image	list[int]	原始图像数据

19.2.5 MapMetaData —地图元数据结构体

字段名	类型	描述
resolution	float	地图分辨率, 单位 m/pixel
origin	Pose3DEuler	地图原点
map_image_data	MapImageData	地图图像数据

19.2.6 MapInfo —单张地图信息结构体

字段名	类型	描述
map_name	str	地图名称
map_meta_data	MapMetaData	地图元数据

19.2.7 AllMapInfo —地图集合结构体

字段名	类型	描述
current_map_name	str	当前地图名称
map_infos	list[MapInfo]	全部地图信息列表

19.2.8 LocalizationInfo —定位信息结构体

字段名	类型	描述
is_localization	bool	是否已完成定位
pose	Pose3DEuler	当前位姿

19.2.9 NavTarget —导航目标点结构体

字段名	类型	描述
id	int	目标点 ID
frame_id	str	目标点坐标系 ID
goal	Pose3DEuler	目标点位姿

19.2.10 NavStatusType —导航状态枚举

枚举值	数值	描述
NONE	0	无状态
RUNNING	1	导航进行中
END_SUCCESS	2	导航成功结束
END_FAILED	3	导航失败结束
PAUSE	4	导航暂停
CONTINUE	5	导航继续
CANCEL	6	导航取消

19.2.11 NavStatus —导航状态结构体

字段名	类型	描述
id	int	目标点 ID, -1 表示当前没有目标点
status	NavStatusType	导航状态
error_code	int	导航错误码
error_desc	str	导航错误描述

19.2.12 Odometry —里程计数据结构体

字段名	类型	描述
header	Header	消息头
child_frame_id	str	子坐标系 ID
position	list[float]	位置坐标 [x, y, z]
orientation	list[float]	四元数姿态 [w, x, y, z]
linear_velocity	list[float]	线速度 [x, y, z]
angular_velocity	list[float]	角速度 [x, y, z]

CHAPTER 20

高层运动控制示例

该示例展示了如何使用 Magicdog-Y1 SDK 进行初始化、连接机器人、高层运动控制（步态、特技、遥控）等基本操作。

20.1 C++:

示例文件: high_level_motion_example.cpp

参考文档: high_level_motion_reference.md

```
#include "magic_robot.h"

#include <termios.h>
#include <unistd.h>
#include <csignal>

#include <iostream>

using namespace magic::y1;

magic::y1::MagicRobot robot;

void signalHandler(int signum) {
    std::cout << "Interrupt signal ( " << signum << " ) received.\n";
```

(下页继续)

(续上页)

```

robot.Shutdown();
// 退出进程
exit(signum);
}

void print_help(const char* prog_name) {
    std::cout << "按键功能演示程序\n\n";
    std::cout << "用法: " << prog_name << "\n";
    std::cout << "按键功能说明:\n";
    std::cout << "  ESC      退出程序\n";
    std::cout << "  1       功能1:锁定站立\n";
    std::cout << "  2       功能2:平衡站立\n";
    std::cout << "  3       功能3:执行特技-打招呼动作\n";
    std::cout << "  w       功能4:向前移动\n";
    std::cout << "  a       功能5:向左移动\n";
    std::cout << "  s       功能6:向后移动\n";
    std::cout << "  d       功能7:向右移动\n";
    std::cout << "  x       功能8:停止移动\n";
    std::cout << "  t       功能9:左转向\n";
    std::cout << "  g       功能10:右转向\n";
}

int getch() {
    struct termios oldt, newt;
    int ch;
    tcgetattr(STDIN_FILENO, &oldt); // 获取当前终端设置
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO); // 关闭缓冲和回显
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    ch = getchar(); // 读取按键
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt); // 恢复设置
    return ch;
}

void RecoveryStand() {
    // 获取高层运控控制器
    auto& controller = robot.GetHighLevelMotionController();

    // 设置步态
    auto status = controller.SetMotionId(MotionId::MOTION_RECOVER_STAND);
    if (status.code != ErrorCode::OK) {
        std::cerr << "set robot gait failed"
            << ", code: " << status.code

```

(下页继续)

(续上页)

```

        << ", message: " << status.message << std::endl;

        return;
    }
}

void BalanceStand() {
    // 获取高层运控控制器
    auto& controller = robot.GetHighLevelMotionController();

    // 设置姿态展示步态
    auto status = controller.SetMotionId(MotionId::MOTION_CROUCH);
    if (status.code != ErrorCode::OK) {
        std::cerr << "set robot gait failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }
    std::cout << "robot motion set to MOTION_CROUCH successfully." << std::endl;
}

void ExecuteTrick() {
    // 获取高层运控控制器
    auto& controller = robot.GetHighLevelMotionController();

    // 执行特技
    auto status = controller.ExecuteTrick(TrickAction::ACTION_SHAKE_LEFT_HAND_REACHOUT);
    if (status.code != ErrorCode::OK) {
        std::cerr << "execute robot trick failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }
    std::cout << "robot trick executed successfully." << std::endl;
}

void JoyStickCommand(float left_x_axis,
                    float left_y_axis,
                    float right_x_axis,
                    float right_y_axis) {
    // 获取高层运控控制器
    auto& controller = robot.GetHighLevelMotionController();

    JoystickCommand joy_command;

```

(下页继续)

(续上页)

```

joy_command.left_x_axis = left_x_axis;
joy_command.left_y_axis = left_y_axis;
joy_command.right_x_axis = right_x_axis;
joy_command.right_y_axis = right_y_axis;
controller.SendJoyStickCommand(joy_command);
}

int main(int argc, char* argv[]) {
    // 绑定 SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    print_help(argv[0]);

    std::string local_ip = "192.168.54.111";
    // 配置本机网线直连机器的IP地址, 并进行SDK初始化
    if (!robot.Initialize(local_ip)) {
        std::cerr << "robot sdk initialize failed." << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 设置rpc超时时间为5s
    robot.SetTimeout(5000);

    // 连接机器人
    auto status = robot.Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "connect robot failed"
                    << ", code: " << status.code
                    << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 切换运控控制器为底层控制器, 默认是高层控制器
    status = robot.SetMotionControlLevel(ControllerLevel::HighLevel);
    if (status.code != ErrorCode::OK) {
        std::cerr << "switch robot motion control level failed"
                    << ", code: " << status.code
                    << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }
}

```

(下页继续)

(续上页)

```
std::cout << "按任意键继续 (ESC退出)..."
    << std::endl;

// 等待用户输入
while (1) {
    int key = getch();
    if (key == 27)
        break; // ESC键ASCII码为27

    std::cout << "按键ASCII: " << key << ", 字符: " << static_cast<char>(key) << std::endl;
    switch (key) {
        case '1': {
            RecoveryStand();
            break;
        }
        case '2': {
            BalanceStand();
            break;
        }
        case '3': {
            ExecuteTrick();
            break;
        }
        case 'w': {
            JoyStickCommand(0.0, 1.0, 0.0, 0.0); // 向前
            break;
        }
        case 'a': {
            JoyStickCommand(-1.0, 0.0, 0.0, 0.0); // 向左
            break;
        }
        case 's': {
            JoyStickCommand(0.0, -1.0, 0.0, 0.0); // 向后
            break;
        }
        case 'd': {
            JoyStickCommand(1.0, 0.0, 0.0, 0.0); // 向右
            break;
        }
        case 'x': {
            JoyStickCommand(0.0, 0.0, 0.0, 0.0); // 停止
            break;
        }
    }
}
```

(下页继续)

(续上页)

```

    }

    case 't': {
        JoyStickCommand(0.0, 0.0, -1.0, 1.0); // 左转
        break;
    }

    case 'g': {
        JoyStickCommand(0.0, 0.0, 1.0, 1.0); // 右转
        break;
    }

    default:
        std::cout << "未知按键: " << key << std::endl;
        break;
    }
}

// 断开与机器人的链接
status = robot.Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;
    robot.Shutdown();
    return -1;
}

robot.Shutdown();

return 0;
}

```

20.2 Python

示例文件: high_level_motion_example.py

参考文档: high_level_motion_reference.md

20.3 完整代码示例

```

#!/usr/bin/env python3

import sys
import time

```

(下页继续)

(续上页)

```
import signal
import logging
from typing import Optional

import magicdog_y1_python as magicbot

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global variables
robot: Optional[magicbot.MagicRobot] = None
running = True

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global running, robot
    logging.info("Received interrupt signal (%s), exiting...", signum)
    running = False
    if robot:
        robot.disconnect()
        logging.info("Robot disconnected")
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

def print_help():
    """Print help information"""
    logging.info("High-Level Motion Control Function Demo Program")
    logging.info("")
    logging.info("Key Function Description:")
    logging.info("  ESC      Exit program")
    logging.info("  1        Function 1: Recovery stand")
    logging.info("  2        Function 2: Balance stand")
    logging.info("  3        Function 3: Execute trick - welcome action")
    logging.info("  w        Function 4: Move forward")
    logging.info("  a        Function 5: Move left")
    logging.info("  s        Function 6: Move backward")
```

(下页继续)

(续上页)

```

logging.info(" d      Function 7: Move right")
logging.info(" x      Function 8: stop move")
logging.info(" t      Function 9: Turn left")
logging.info(" g      Function 10: Turn right")

def get_user_input():
    """Get user input"""
    try:
        # Python implementation of getch() on Linux systems
        import tty
        import termios

        fd = sys.stdin.fileno()
        old_settings = termios.tcgetattr(fd)

        try:
            tty.setraw(sys.stdin.fileno())
            ch = sys.stdin.read(1)

        finally:
            termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)

        return ch
    except ImportError:
        # If termios is not available, use simple input
        return input("Please press a key: ").strip()

def recovery_stand():
    """Recovery stand"""
    global robot
    try:
        logging.info("=== Executing Recovery Stand ===")

        # Get high-level motion controller
        controller = robot.get_high_level_motion_controller()

        # Set gait to recovery stand
        status = controller.set_motion_id(magicbot.MotionId.MOTION_RECOVER_STAND)
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to set robot gait, code: %s, message: %s",
                status.code,
                status.message,
            )
    
```

(下页继续)

(续上页)

```

        return False

    logging.info("Robot gait set to recovery stand")
    return True

except Exception as e:
    logging.error("Exception occurred while executing recovery stand: %s", e)
    return False

def balance_stand():
    """Balance stand"""
    global robot
    try:
        logging.info("=== Executing Balance Stand ===")

        # Get high-level motion controller
        controller = robot.get_high_level_motion_controller()

        # Set gait to balance stand
        status = controller.set_motion_id(magicbot.MotionId.MOTION_CROUCH)
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to set robot gait, code: %s, message: %s",
                status.code,
                status.message,
            )
            return False

        logging.info("Robot gait set to balance stand (supports movement)")
        return True

    except Exception as e:
        logging.error("Exception occurred while executing balance stand: %s", e)
        return False

def execute_trick_welcome():
    """Execute trick - welcome action"""
    global robot
    try:
        logging.info("=== Executing Trick - Celebrate Action ===")

```

(下页继续)

(续上页)

```

    # Get high-level motion controller
    controller = robot.get_high_level_motion_controller()

    # Execute welcome trick
    status = controller.execute_trick(magicbot.TrickAction.ACTION_WELCOME)
    if status.code != magicbot.ErrorCode.OK:
        logging.error(
            "Failed to execute robot trick, code: %s, message: %s",
            status.code,
            status.message,
        )
        return False

    logging.info("Robot trick executed successfully")
    return True

except Exception as e:
    logging.error("Exception occurred while executing trick: %s", e)
    return False

def joystick_command(left_x_axis, left_y_axis, right_x_axis, right_y_axis):
    """Send joystick control command"""
    global robot
    try:
        # Get high-level motion controller
        controller = robot.get_high_level_motion_controller()

        # Create joystick command
        joy_command = magicbot.JoystickCommand()
        joy_command.left_x_axis = left_x_axis
        joy_command.left_y_axis = left_y_axis
        joy_command.right_x_axis = right_x_axis
        joy_command.right_y_axis = right_y_axis

        # Send joystick command
        controller.send_joystick_command(joy_command)
    except Exception as e:
        logging.error("Exception occurred while sending joystick command: %s", e)

def move_forward():
    """Move forward"""

```

(下页继续)

(续上页)

```
logging.info("=== Moving Forward ===")
return joystick_command(0.0, 1.0, 0.0, 0.0)

def move_backward():
    """Move backward"""
    logging.info("=== Moving Backward ===")
    return joystick_command(0.0, -1.0, 0.0, 0.0)

def move_left():
    """Move left"""
    logging.info("=== Moving Left ===")
    return joystick_command(-1.0, 0.0, 0.0, 0.0)

def move_right():
    """Move right"""
    logging.info("=== Moving Right ===")
    return joystick_command(1.0, 0.0, 0.0, 0.0)

def turn_left():
    """Turn left"""
    logging.info("=== Turning Left ===")
    return joystick_command(0.0, 0.0, -1.0, 0.0)

def turn_right():
    """Turn right"""
    logging.info("=== Turning Right ===")
    return joystick_command(0.0, 0.0, 1.0, 0.0)

def stop_move():
    """Stop move"""
    logging.info("=== Stopping Move ===")
    return joystick_command(0.0, 0.0, 0.0, 0.0)

def main():
    """Main function"""
    global robot, running
```

(下页继续)

(续上页)

```
# Bind signal handler
signal.signal(signal.SIGINT, signal_handler)

logging.info("Robot model: %s", magicbot.get_robot_model())

# Create robot instance
robot = magicbot.MagicRobot()

try:
    # Configure local IP address for direct network connection and initialize SDK
    local_ip = "192.168.54.111"
    if not robot.initialize(local_ip):
        logging.error("Failed to initialize robot SDK")
        robot.shutdown()
        return -1

    # Set RPC timeout to 5 seconds
    robot.set_timeout(5000)

    # Connect to robot
    status = robot.connect()
    if status.code != magicbot.ErrorCode.OK:
        logging.error(
            "Failed to connect to robot, code: %s, message: %s",
            status.code,
            status.message,
        )
        robot.shutdown()
        return -1

    logging.info("Successfully connected to robot")

    # Switch motion control controller to high-level controller
    status = robot.set_motion_control_level(magicbot.ControllerLevel.HighLevel)
    if status.code != magicbot.ErrorCode.OK:
        logging.error(
            "Failed to switch robot motion control level, code: %s, message: %s",
            status.code,
            status.message,
        )
        robot.shutdown()
        return -1
```

(下页继续)

(续上页)

```

logging.info("Switched to high-level motion controller")

# Initialize high-level motion controller
controller = robot.get_high_level_motion_controller()
if not controller.initialize():
    logging.error("Failed to initialize high-level motion controller")
    robot.disconnect()
    robot.shutdown()
    return -1

logging.info("Successfully initialized high-level motion controller")

print_help()
logging.info("Press any key to continue (ESC to exit)...")

# Main loop
while running:
    try:
        key = get_user_input()

        if key == "\x1b": # ESC key
            break

        logging.info("Key pressed: %s", key)

        if key == "1":
            recovery_stand()
        elif key == "2":
            balance_stand()
        elif key == "3":
            execute_trick_welcome()
        elif key == "w":
            move_forward()
        elif key == "a":
            move_left()
        elif key == "s":
            move_backward()
        elif key == "d":
            move_right()
        elif key == "x":
            stop_move()
        elif key == "t":

```

(下页继续)

(续上页)

```

        turn_left()
    elif key == "q":
        turn_right()
    else:
        logging.info("Unknown key: %s", key)

    time.sleep(0.01)  # Brief delay

except KeyboardInterrupt:
    break
except Exception as e:
    logging.error("Exception occurred while processing user input: %s", e)

except Exception as e:
    logging.error("Exception occurred during program execution: %s", e)
    return -1

finally:
    # Clean up resources
    try:
        logging.info("Clean up resources")
        # Close high-level motion controller
        controller = robot.get_high_level_motion_controller()
        controller.shutdown()
        logging.info("High-level motion controller closed")

        # Disconnect
        robot.disconnect()
        logging.info("Robot connection disconnected")

        # Shutdown robot
        robot.shutdown()
        logging.info("Robot shutdown")

    except Exception as e:
        logging.error("Exception occurred while cleaning up resources: %s", e)

if __name__ == "__main__":
    sys.exit(main())

```

20.4 运行说明

1. 环境准备：

```
# 设置环境变量
export PYTHONPATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$LD_LIBRARY_PATH
```

2. 运行示例：

```
# C++
./high_level_motion_example
# Python
python3 high_level_motion_example.py
```

3. 控制说明：

- ESC：退出程序
- 1：恢复站立
- 2：平衡站立
- 3：庆祝动作
- w/a/s/d/x：前进/左移/后退/右移/停止
- t/g：左转/右转

4. 停止程序：

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

CHAPTER 21

底层运动控制示例

该示例展示了如何使用 Magicdog-Y1 SDK 进行初始化、连接机器人、底层运动控制（手、臂、腿、腰、头）、Imu 传感器数据读取等基本操作。

21.1 C++:

示例文件: low_level_motion_example.cpp

参考文档: low_level_motion_reference.md

```
#include "magic_robot.h"

#include <unistd.h>
#include <csignal>

#include <iostream>

using namespace magic::y1;

magic::y1::MagicRobot robot;

std::atomic<bool> running(true);

void signalHandler(int signum) {
    std::cout << "Interrupt signal (" << signum << ") received.\n";
```

(下页继续)

(续上页)

```

    running = false;

    robot.Shutdown();
    // 退出进程
    exit(signum);
}

int main() {
    // 绑定 SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    std::string local_ip = "192.168.54.111";
    // 配置本机网线直连机器的IP地址, 并进行SDK初始化
    if (!robot.Initialize(local_ip)) {
        std::cerr << "robot sdk initialize failed." << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 连接机器人
    auto status = robot.Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "connect robot failed"
                    << ", code: " << status.code
                    << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 切换运控控制器为底层控制器, 默认是高层控制器
    status = robot.SetMotionControlLevel(ControllerLevel::LowLevel);
    if (status.code != ErrorCode::OK) {
        std::cerr << "switch robot motion control level failed"
                    << ", code: " << status.code
                    << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 获取底层控制器
    auto& controller = robot.GetLowLevelMotionController();

```

(下页继续)

(续上页)

```

// 订阅imu数据
controller.SubscribeBodyImu([](const std::shared_ptr<Imu> msg) {
    static int32_t count = 0;
    if (count++ % 1000 == 1) {
        std::cout << "+++++++ receive imu data." << std::endl;
        std::cout << "timestamp: " << msg->timestamp << std::endl;
        std::cout << "temperature: " << msg->temperature << std::endl;
        std::cout << "orientation: " << msg->orientation[0] << ", " << msg->orientation[1] << ",
↪" << msg->orientation[2] << ", " << msg->orientation[3] << std::endl;
        std::cout << "angular_velocity: " << msg->angular_velocity[0] << ", " << msg->angular_
↪velocity[1] << ", " << msg->angular_velocity[2] << std::endl;
        std::cout << "linear_acceleration: " << msg->linear_acceleration[0] << ", " << msg->
↪linear_acceleration[1] << ", " << msg->linear_acceleration[2] << std::endl;
    }
    // TODO: handle imu data
});

// 订阅手部数据
controller.SubscribeArmState([](const std::shared_ptr<JointState> msg) {
    static int32_t count = 0;
    if (count++ % 1000 == 1) {
        std::cout << "+++++++ receive arm joint data." << std::endl;
        std::cout << "timestamp: " << msg->timestamp << std::endl;
        std::cout << "pos: " << msg->joints[0].posH << ", " << msg->joints[0].posL << std::endl;
        std::cout << "vel: " << msg->joints[0].vel << std::endl;
        std::cout << "toq: " << msg->joints[0].toq << std::endl;
        std::cout << "current: " << msg->joints[0].current << std::endl;
        std::cout << "error_code: " << msg->joints[0].err_code << std::endl;
    }
    // TODO: handle arm joint data
});

// 以上臂关节控制为例：
// 后续关节控制指令，关节的操作模式为1，表示关节处于位置控制模式
while (running.load()) {
    // 左臂关节，参考文档：
    // 左臂或者右臂1-5关节operation_
↪mode需要从模式：200切换到模式：4（串联PID模式）进行指令下发；
    JointCommand arm_command;
    arm_command.joints.resize(kArmJointNum);
    for (int ii = 0; ii < kArmJointNum; ii++) {
        // 设置关节处于准备状态
        arm_command.joints[ii].operation_mode = 200;
    }
}

```

(下页继续)

(续上页)

```

// TODO:设置目标位置、速度、力矩和增益
arm_command.joints[ii].pos = 0.0;
arm_command.joints[ii].vel = 0.0;
arm_command.joints[ii].toq = 0.0;
arm_command.joints[ii].kp = 0.0;
arm_command.joints[ii].kd = 0.0;
}
// 发布控制指令
controller.PublishArmCommand(arm_command);

// 500HZ的频率(2ms)下发控制指令
usleep(2000);
}

// 断开与机器人的链接
status = robot.Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;
    robot.Shutdown();
    return -1;
}

robot.Shutdown();

return 0;
}

```

21.2 Python

示例文件: low_level_motion_example.py

参考文档: low_level_motion_reference.md

21.3 完整代码示例

```

#!/usr/bin/env python3

import sys
import time
import signal

```

(下页继续)

(续上页)

```
import logging
from typing import Optional

import magicdog_y1_python as magicbot

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global variables
robot: Optional[magicbot.MagicRobot] = None
running = True

body_imu_counter = 0
arm_state_counter = 0
leg_state_counter = 0
head_state_counter = 0
waist_state_counter = 0

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global robot, running
    running = False
    logging.info("Received interrupt signal (%s), exiting...", signum)

    if robot:
        robot.disconnect()
        logging.info("Robot disconnected")
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

def body_imu_callback(imu_data):
    """Body IMU data callback function"""
    global body_imu_counter
    if body_imu_counter % 1000 == 0:
        logging.info("+++++++ Received body IMU data")
        # Print IMU data
```

(下页继续)

(续上页)

```

logging.info("Received body IMU data, timestamp: %d", imu_data.timestamp)
logging.info("Received body IMU data, orientation: %s", imu_data.orientation)
logging.info(
    "Received body IMU data, angular_velocity: %s",
    imu_data.angular_velocity,
)
logging.info(
    "Received body IMU data, linear_acceleration: %s",
    imu_data.linear_acceleration,
)
logging.info("Received body IMU data, temperature: %s", imu_data.temperature)
body_imu_counter += 1

def arm_state_callback(joint_state):
    """Arm joint state callback function"""
    global arm_state_counter
    if arm_state_counter % 1000 == 0:
        logging.info("+++++++ Received arm joint state data")
        # Print joint state data
        logging.info(
            "Received arm joint state data, status_word: %d",
            joint_state.joints[0].status_word,
        )
        logging.info(
            "Received arm joint state data, posH: %s", joint_state.joints[0].posH
        )
        logging.info(
            "Received arm joint state data, posL: %s", joint_state.joints[0].posL
        )
        logging.info(
            "Received arm joint state data, vel: %s", joint_state.joints[0].vel
        )
        logging.info(
            "Received arm joint state data, toq: %s", joint_state.joints[0].toq
        )
        logging.info(
            "Received arm joint state data, current: %s", joint_state.joints[0].current
        )
        logging.info(
            "Received arm joint state data, error_code: %s",
            joint_state.joints[0].err_code,
        )

```

(下页继续)

(续上页)

```

arm_state_counter += 1

def leg_state_callback(joint_state):
    """Leg joint state callback function"""
    global leg_state_counter
    if leg_state_counter % 1000 == 0:
        logging.info("+++++++ Received leg joint state data")
        # Print joint state data
        logging.info(
            "Received leg joint state data, status_word: %d",
            joint_state.joints[0].status_word,
        )
        logging.info(
            "Received leg joint state data, posH: %s", joint_state.joints[0].posH
        )
        logging.info(
            "Received leg joint state data, posL: %s", joint_state.joints[0].posL
        )
        logging.info(
            "Received leg joint state data, vel: %s", joint_state.joints[0].vel
        )
        logging.info(
            "Received leg joint state data, toq: %s", joint_state.joints[0].toq
        )
        logging.info(
            "Received leg joint state data, current: %s", joint_state.joints[0].current
        )
        logging.info(
            "Received leg joint state data, error_code: %s",
            joint_state.joints[0].err_code,
        )
    leg_state_counter += 1

def head_state_callback(joint_state):
    """Head joint state callback function"""
    global head_state_counter
    if head_state_counter % 1000 == 0:
        logging.info("+++++++ Received head joint state data")
        # Print joint state data
        logging.info(
            "Received head joint state data, status_word: %d",

```

(下页继续)

(续上页)

```

        joint_state.joints[0].status_word,
    )
    logging.info(
        "Received head joint state data, posH: %s", joint_state.joints[0].posH
    )
    logging.info(
        "Received head joint state data, posL: %s", joint_state.joints[0].posL
    )
    logging.info(
        "Received head joint state data, vel: %s", joint_state.joints[0].vel
    )
    logging.info(
        "Received head joint state data, toq: %s", joint_state.joints[0].toq
    )
    logging.info(
        "Received head joint state data, current: %s", joint_state.joints[0].current
    )
    logging.info(
        "Received head joint state data, error_code: %s",
        joint_state.joints[0].err_code,
    )
    head_state_counter += 1

def waist_state_callback(joint_state):
    """Waist joint state callback function"""
    global waist_state_counter
    if waist_state_counter % 1000 == 0:
        logging.info("+++++++ Received waist joint state data")
        # Print joint state data
        logging.info(
            "Received waist joint state data, status_word: %d",
            joint_state.joints[0].status_word,
        )
        logging.info(
            "Received waist joint state data, posH: %s", joint_state.joints[0].posH
        )
        logging.info(
            "Received waist joint state data, posL: %s", joint_state.joints[0].posL
        )
        logging.info(
            "Received waist joint state data, vel: %s", joint_state.joints[0].vel
        )

```

(下页继续)

(续上页)

```

        logging.info(
            "Received waist joint state data, toq: %s", joint_state.joints[0].toq
        )
        logging.info(
            "Received waist joint state data, current: %s",
            joint_state.joints[0].current,
        )
        logging.info(
            "Received waist joint state data, error_code: %s",
            joint_state.joints[0].err_code,
        )
        waist_state_counter += 1

def main():
    """Main function"""
    global robot

    # Bind signal handler
    signal.signal(signal.SIGINT, signal_handler)

    logging.info("Robot model: %s", magicbot.get_robot_model())

    # Create robot instance
    robot = magicbot.MagicRobot()

    try:
        # Configure local IP address for direct network connection and initialize SDK
        local_ip = "192.168.54.111"
        if not robot.initialize(local_ip):
            logging.error("Failed to initialize robot SDK")
            robot.shutdown()
            return -1

        # Connect to robot
        status = robot.connect()
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to connect to robot, code: %s, message: %s",
                status.code,
                status.message,
            )
            robot.shutdown()
    
```

(下页继续)

(续上页)

```

        return -1

    logging.info("Successfully connected to robot")

    # Switch motion control controller to low-level controller, default is high-level_
↪controller
    status = robot.set_motion_control_level(magicbot.ControllerLevel.LowLevel)
    if status.code != magicbot.ErrorCode.OK:
        logging.error(
            "Failed to switch robot motion control level, code: %s, message: %s",
            status.code,
            status.message,
        )
        robot.shutdown()
        return -1

    logging.info("Switched to low-level motion controller")

    # Get low-level motion controller
    controller = robot.get_low_level_motion_controller()

    # Subscribe to body IMU data
    controller.subscribe_body_imu(body_imu_callback)
    logging.info("Subscribed to body IMU data")

    # Subscribe to arm joint state
    controller.subscribe_arm_state(arm_state_callback)
    logging.info("Subscribed to arm joint state")

    # Subscribe to leg joint state
    controller.subscribe_leg_state(leg_state_callback)
    logging.info("Subscribed to leg joint state")

    # Subscribe to head joint state
    controller.subscribe_head_state(head_state_callback)
    logging.info("Subscribed to head joint state")

    # Subscribe to waist joint state
    controller.subscribe_waist_state(waist_state_callback)
    logging.info("Subscribed to waist joint state")

    # Main loop
    global running

```

(下页继续)

(续上页)

```
while running:
    # Create arm joint control command
    arm_command = magicbot.JointCommand()
    leg_command = magicbot.JointCommand()
    waist_command = magicbot.JointCommand()
    head_command = magicbot.JointCommand()

    # Set all joints to preparation state (operation mode 200)
    for i in range(magicbot.ARM_JOINT_NUM):
        joint = magicbot.SingleJointCommand()
        joint.operation_mode = 200 # Preparation state
        joint.pos = 0.0
        joint.vel = 0.0
        joint.toq = 0.0
        joint.kp = 0.0
        joint.kd = 0.0
        arm_command.joints.append(joint)

    for i in range(magicbot.LEG_JOINT_NUM):
        joint = magicbot.SingleJointCommand()
        joint.operation_mode = 200 # Preparation state
        joint.pos = 0.0
        joint.vel = 0.0
        joint.toq = 0.0
        joint.kp = 0.0
        joint.kd = 0.0
        leg_command.joints.append(joint)

    for i in range(magicbot.HEAD_JOINT_NUM):
        joint = magicbot.SingleJointCommand()
        joint.operation_mode = 200 # Preparation state
        joint.pos = 0.0
        joint.vel = 0.0
        joint.toq = 0.0
        joint.kp = 0.0
        joint.kd = 0.0
        head_command.joints.append(joint)

    for i in range(magicbot.WAIST_JOINT_NUM):
        joint = magicbot.SingleJointCommand()
        joint.operation_mode = 200 # Preparation state
        joint.pos = 0.0
        joint.vel = 0.0
```

(下页继续)

(续上页)

```

        joint.toq = 0.0
        joint.kp = 0.0
        joint.kd = 0.0
        waist_command.joints.append(joint)

        # Publish arm joint control command
        controller.publish_arm_command(arm_command)

        # Publish leg joint control command
        controller.publish_leg_command(leg_command)

        # Publish waist joint control command
        controller.publish_waist_command(waist_command)

        # Publish head joint control command
        controller.publish_head_command(head_command)

        time.sleep(0.002)

    except Exception as e:
        logging.error("Exception occurred during program execution: %s", e)
        return -1

    finally:
        # Clean up resources
        try:
            logging.info("Clean up resources")
            # Close low-level motion controller
            controller = robot.get_low_level_motion_controller()
            controller.shutdown()
            logging.info("Low-level motion controller closed")

            # Disconnect
            robot.disconnect()
            logging.info("Robot connection disconnected")

            # Shutdown robot
            robot.shutdown()
            logging.info("Robot shutdown")

        except Exception as e:
            logging.error("Exception occurred while cleaning up resources: %s", e)

```

(下页继续)

(续上页)

```
if __name__ == "__main__":  
    sys.exit(main())
```

21.4 运行说明

1. 环境准备:

```
# 设置环境变量  
export PYTHONPATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$PYTHONPATH  
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$LD_LIBRARY_PATH
```

2. 运行示例:

```
# C++  
./low_level_motion_example  
  
# Python  
python3 low_level_motion_example.py
```

3. 停止程序:

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

传感器控制示例

该示例展示了如何使用 Magicdog-Y1 SDK 进行初始化、连接机器人、机器人传感器控制（雷达、RGBD 相机、双目相机）等基本操作。

22.1 C++

示例文件：sensor_example.cpp

参考文档：sensor_reference.md

```
#include "magic_robot.h"
#include "magic_type.h"

#include <unistd.h>
#include <csignal>

#include <iostream>

using namespace magic::y1;

magic::y1::MagicRobot robot;

void signalHandler(int signum) {
    std::cout << "Interrupt signal (" << signum << ") received.\n";
```

(下页继续)

(续上页)

```
robot.Shutdown();
// 退出进程
exit(signum);
}

int main() {
    // 绑定 SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    std::string local_ip = "192.168.54.111";
    // 配置本机网线直连机器的IP地址, 并进行SDK初始化
    if (!robot.Initialize(local_ip)) {
        std::cerr << "robot sdk initialize failed." << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 设置rpc超时时间为5s
    robot.SetTimeout(5000);

    // 连接机器人
    auto status = robot.Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "connect robot failed"
                    << ", code: " << status.code
                    << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    auto& controller = robot.GetSensorController();

    // 打开lidar
    status = controller.OpenLidar();
    if (status.code != ErrorCode::OK) {
        std::cerr << "open lidar failed"
                    << ", code: " << status.code
                    << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 订阅Lidar Imu数据
```

(下页继续)

(续上页)

```

controller.SubscribeLidarImu([] (const std::shared_ptr<Imu> msg) {
    std::cout << "+++++++ receive lidar imu." << std::endl;

    std::cout << "lidar imu timestamp: " << msg->timestamp << std::endl;
    std::cout << "lidar imu orientation: " << msg->orientation[0] << ", " << msg->
orientation[1] << ", " << msg->orientation[2] << ", " << msg->orientation[3] << std::endl;
    std::cout << "lidar imu angular_velocity: " << msg->angular_velocity[0] << ", " << msg->
angular_velocity[1] << ", " << msg->angular_velocity[2] << std::endl;
    std::cout << "lidar imu linear_acceleration: " << msg->linear_acceleration[0] << ", " <<
msg->linear_acceleration[1] << ", " << msg->linear_acceleration[2] << std::endl;
    std::cout << "lidar imu temperature: " << msg->temperature << std::endl;
});

// 订阅Lidar PointCloud数据
controller.SubscribeLidarPointCloud([] (const std::shared_ptr<PointCloud2> msg) {
    std::cout << "+++++++ receive lidar point cloud." << std::endl;
    // TODO: handle lidar point cloud data
    std::cout << "lidar point cloud size: " << msg->data.size() << std::endl;
    std::cout << "lidar point cloud width: " << msg->width << std::endl;
    std::cout << "lidar point cloud height: " << msg->height << std::endl;
    std::cout << "lidar point cloud is_dense: " << msg->is_dense << std::endl;
    std::cout << "lidar point cloud is_bigendian: " << msg->is_bigendian << std::endl;
    std::cout << "lidar point cloud point_step: " << msg->point_step << std::endl;
    std::cout << "lidar point cloud row_step: " << msg->row_step << std::endl;
    std::cout << "lidar point cloud fields: " << msg->fields.size() << std::endl;
    std::cout << "lidar point cloud fields name: " << msg->fields[0].name << std::endl;
    std::cout << "lidar point cloud fields offset: " << msg->fields[0].offset << std::endl;
    std::cout << "lidar point cloud fields datatype: " << msg->fields[0].datatype << std::endl;
    std::cout << "lidar point cloud fields count: " << msg->fields[0].count << std::endl;
});

usleep(20000000);

// 关闭lidar
status = controller.CloseLidar();
if (status.code != ErrorCode::OK) {
    std::cerr << "close lidar failed"
        << ", code: " << status.code
        << ", message: " << status.message << std::endl;
    robot.Shutdown();
    return -1;
}
    
```

(下页继续)

(续上页)

```
// 断开与机器人的链接
status = robot.Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;
    robot.Shutdown();
    return -1;
}

robot.Shutdown();

return 0;
}
```

22.2 Python

示例文件: sensor_example.py

参考文档: sensor_reference.md

22.3 完整代码示例

```
#!/usr/bin/env python3

import sys
import time
import signal
import logging
from typing import Optional

import magicdog_y1_python as magicbot

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global variables
```

(下页继续)

(续上页)

```
robot: Optional[magicbot.MagicRobot] = None

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global robot
    logging.info(f"Received interrupt signal ({signum}), exiting...")
    if robot:
        robot.disconnect()
        logging.info("Robot disconnected")
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

imu_counter = 0
def lidar_imu_callback(imu_data):
    """LiDAR IMU data callback function"""
    global imu_counter
    imu_counter += 1
    if imu_counter % 100 == 0:
        logging.info("+++++++ Received LiDAR IMU data")
        logging.info(
            "Received LiDAR IMU data, counter: %d, timestamp: %d",
            imu_counter,
            imu_data.timestamp,
        )
        logging.info("Received LiDAR IMU data, orientation: %s", imu_data.orientation)
        logging.info(
            "Received LiDAR IMU data, angular_velocity: %s", imu_data.angular_velocity
        )
        logging.info(
            "Received LiDAR IMU data, linear_acceleration: %s",
            imu_data.linear_acceleration,
        )
        logging.info("Received LiDAR IMU data, temperature: %s", imu_data.temperature)

point_cloud_counter = 0
def lidar_point_cloud_callback(point_cloud_data):
    """LiDAR point cloud data callback function"""
    global point_cloud_counter
    point_cloud_counter += 1
```

(下页继续)

(续上页)

```

logging.info("Received LiDAR point cloud data, counter: %d", point_cloud_counter)

def main():
    """Main function"""
    global robot

    # Bind signal handler
    signal.signal(signal.SIGINT, signal_handler)

    logging.info("Robot model: %s", magicbot.get_robot_model())

    # Create robot instance
    robot = magicbot.MagicRobot()

    try:
        # Configure local IP address for direct network connection and initialize SDK
        local_ip = "192.168.54.111"
        if not robot.initialize(local_ip):
            logging.error("Failed to initialize robot SDK")
            robot.shutdown()
            return -1

        # Connect to robot
        status = robot.connect()
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to connect to robot, code: %s, message: %s",
                status.code,
                status.message,
            )
            robot.shutdown()
            return -1

        logging.info("Successfully connected to robot")

        # Get sensor controller
        controller = robot.get_sensor_controller()

        # Open LiDAR
        status = controller.open_lidar()
        if status.code != magicbot.ErrorCode.OK:
            logging.error(

```

(下页继续)

(续上页)

```

        "Failed to open LiDAR, code: %s, message: %s",
        status.code,
        status.message,
    )
    robot.shutdown()
    return -1

logging.info("LiDAR opened successfully")

# Subscribe to LiDAR IMU data
controller.subscribe_lidar_imu(lidar_imu_callback)
logging.info("Subscribed to LiDAR IMU data")

# Subscribe to LiDAR point cloud data
controller.subscribe_lidar_point_cloud(lidar_point_cloud_callback)
logging.info("Subscribed to LiDAR point cloud data")

# Wait 20 seconds to receive data
logging.info("Waiting 20 seconds to receive sensor data...")
time.sleep(20)

# Close LiDAR
status = controller.close_lidar()
if status.code != magicbot.ErrorCode.OK:
    logging.error(
        "Failed to close LiDAR, code: %s, message: %s",
        status.code,
        status.message,
    )
    robot.shutdown()
    return -1

logging.info("LiDAR closed successfully")

except Exception as e:
    logging.error("Exception occurred during program execution: %s", e)
    return -1

finally:
    # Clean up resources
    try:
        logging.info("Clean up resources")
        # Close sensor controller

```

(下页继续)

(续上页)

```

        sensor_controller = robot.get_sensor_controller()
        sensor_controller.shutdown()
        logging.info("Sensor controller closed")

        # Disconnect
        robot.disconnect()
        logging.info("Robot connection disconnected")

        # Shutdown robot
        robot.shutdown()
        logging.info("Robot shutdown")

    except Exception as e:
        logging.error("Exception occurred while cleaning up resources: %s", e)

if __name__ == "__main__":
    sys.exit(main())

```

22.4 运行说明

1. 环境准备:

```

# 设置环境变量
export PYTHONPATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$LD_LIBRARY_PATH

```

2. 运行示例:

```

# C++
./sensor_example

# Python
python3 sensor_example.py

```

3. 停止程序:

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

语音控制示例

该示例展示了如何使用 Magicdog-Y1 SDK 进行初始化、连接机器人、音频控制（获取音量、设置音量、TTS 播放、音频流订阅）等基本操作。

23.1 C++

示例文件: `audio_example.cpp`

参考文档: `audio_reference.md`

```
#include "magic_robot.h"

#include <termios.h>
#include <unistd.h>
#include <csignal>

#include <iostream>

using namespace magic::y1;

magic::y1::MagicRobot robot;

void signalHandler(int signum) {
    std::cout << "Interrupt signal (" << signum << ") received.\n";
```

(下页继续)

(续上页)

```
robot.Shutdown();
// 退出进程
exit(signum);
}

void print_help(const char* prog_name) {
    std::cout << "按键功能演示程序\n\n";
    std::cout << "用法: " << prog_name << "\n";
    std::cout << "按键功能说明:\n";
    std::cout << "  ESC      退出程序\n";
    std::cout << "  1       功能1:获取音量\n";
    std::cout << "  2       功能2:设置音量\n";
    std::cout << "  3       功能3:播放语音\n";
    std::cout << "  4       功能4:停止播放\n";
    std::cout << "  5       功能5:打开音频流\n";
    std::cout << "  6       功能6:关闭音频流\n";
    std::cout << "  7       功能7:订阅音频流\n";
}

int getch() {
    struct termios oldt, newt;
    int ch;
    tcgetattr(STDIN_FILENO, &oldt); // 获取当前终端设置
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO); // 关闭缓冲和回显
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    ch = getchar(); // 读取按键
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt); // 恢复设置
    return ch;
}

void GetVolume() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();

    // 获取音量
    int get_volume = 0;
    auto status = controller.GetVolume(get_volume);
    if (status.code != ErrorCode::OK) {
        std::cerr << "get volume failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;
        return;
    }
}
```

(下页继续)

(续上页)

```

    }

    std::cout << "get volume success, volume: " << std::to_string(get_volume) << std::endl;
}

void SetVolume() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();
    // 设置音量
    auto status = controller.SetVolume(7);
    if (status.code != ErrorCode::OK) {
        std::cerr << "set volume failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "set volume success" << std::endl;
}

void PlayTts() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();
    // 播放语音
    TtsCommand tts;
    tts.id = "1000000000001";
    tts.content = "今天天气怎么样! ";
    tts.priority = TtsPriority::HIGH;
    tts.mode = TtsMode::CLEARTOP;
    auto status = controller.Play(tts);
    if (status.code != ErrorCode::OK) {
        std::cerr << "play tts failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "play tts success" << std::endl;
}

void StopTts() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();
    // 停止播放语音
    auto status = controller.Stop();
    if (status.code != ErrorCode::OK) {

```

(下页继续)

(续上页)

```

        std::cerr << "stop tts failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "stop tts success" << std::endl;
}

void OpenAudioStream() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();
    // 打开音频流
    auto status = controller.OpenAudioStream();
    if (status.code != ErrorCode::OK) {
        std::cerr << "open audio stream failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "open audio stream success" << std::endl;
}

void CloseAudioStream() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();
    // 关闭音频流
    auto status = controller.CloseAudioStream();
    if (status.code != ErrorCode::OK) {
        std::cerr << "close audio stream failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "close audio stream success" << std::endl;
}

void SubscribeAudioStream() {
    // 获取音频控制器
    auto& controller = robot.GetAudioController();

    // 订阅音频流
    controller.SubscribeOriginAudioStream([](const std::shared_ptr<AudioStream> data) {
        static int32_t counter = 0;

```

(下页继续)

(续上页)

```

        if (counter++ % 30 == 0) {
            std::cout << "Received origin audio stream data, size: " << data->data_length << "\n";
        }
    });

    controller.SubscribeBfAudioStream([] (const std::shared_ptr<AudioStream> data) {
        static int32_t counter = 0;
        if (counter++ % 30 == 0) {
            std::cout << "Received bf audio stream data, size: " << data->data_length << std::endl;
        }
    });

    std::cout << "Subscribed to audio streams" << std::endl;
}

int main(int argc, char* argv[]) {
    // 绑定 SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    print_help(argv[0]);

    std::string local_ip = "192.168.54.111";
    // 配置本机网线直连机器的IP地址, 并进行SDK初始化
    if (!robot.Initialize(local_ip)) {
        std::cerr << "robot sdk initialize failed." << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 设置rpc超时时间为5s
    robot.SetTimeout(5000);

    // 连接机器人
    auto status = robot.Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "connect robot failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    std::cout << "按任意键继续 (ESC退出)..."

```

(下页继续)

(续上页)

```

        << std::endl;

// 等待用户输入
while (1) {
    int key = getch();
    if (key == 27)
        break; // ESC键ASCII码为27

    std::cout << "按键ASCII: " << key << ", 字符: " << static_cast<char>(key) << std::endl;
    switch (key) {
        case '1': {
            // 获取音量
            GetVolume();
            break;
        }
        case '2': {
            // 设置音量
            SetVolume();
            break;
        }
        case '3': {
            PlayTts();
            break;
        }
        case '4': {
            StopTts();
            break;
        }
        case '5': {
            OpenAudioStream();
            break;
        }
        case '6': {
            CloseAudioStream();
            break;
        }
        case '7': {
            SubscribeAudioStream();
            break;
        }
        default:
            std::cout << "未知按键: " << key << std::endl;
            break;
    }
}

```

(下页继续)

(续上页)

```

    }
}

// 断开与机器人的链接
status = robot.Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed"
                << ", code: " << status.code
                << ", message: " << status.message << std::endl;
    robot.Shutdown();
    return -1;
}

robot.Shutdown();

return 0;
}

```

23.2 Python

示例文件: `audio_example.py`

参考文档: `audio_reference.md`

23.3 完整代码示例

```

#!/usr/bin/env python3

import sys
import time
import signal
import threading
import logging
from typing import Optional

import magicdog_y1_python as magicbot

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",

```

(下页继续)

(续上页)

```

        force=True,
    )

# Global variables
robot: Optional[magicbot.MagicRobot] = None
running = True

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global running, robot
    logging.info("Received interrupt signal (%s), exiting...", signum)
    running = False
    if robot:
        robot.disconnect()
        logging.info("Robot disconnected")
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

def print_help():
    """Print help information"""
    logging.info("Key Function Demo Program")
    logging.info("")
    logging.info("Key Function Description:")
    logging.info("  ESC      Exit program")
    logging.info("  1        Function 1: Get volume")
    logging.info("  2        Function 2: Set volume")
    logging.info("  3        Function 3: Play TTS")
    logging.info("  4        Function 4: Stop playback")
    logging.info("  5        Function 5: Open audio stream")
    logging.info("  6        Function 6: Close audio stream")
    logging.info("  7        Function 7: Subscribe to audio stream")

def get_volume():
    """Get volume"""
    global robot
    try:
        # Get audio controller
        controller = robot.get_audio_controller()

```

(下页继续)

(续上页)

```
# Get volume
status, volume = controller.get_volume()
if status.code != magicbot.ErrorCode.OK:
    logging.error(
        "Failed to get volume, code: %s, message: %s",
        status.code,
        status.message,
    )
    return

logging.info("Successfully got volume, volume: %s", volume)
except Exception as e:
    logging.error("Exception occurred while getting volume: %s", e)

def set_volume():
    """Set volume"""
    global robot
    try:
        # Get audio controller
        controller = robot.get_audio_controller()

        # Set volume to 7
        status = controller.set_volume(7)
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to set volume, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully set volume")
    except Exception as e:
        logging.error("Exception occurred while setting volume: %s", e)

def play_tts():
    """Play TTS speech"""
    global robot
    try:
        # Get audio controller
        controller = robot.get_audio_controller()
```

(下页继续)

(续上页)

```

        # Create TTS command
        tts = magicbot.TtsCommand()
        tts.id = "100000000001"
        tts.content = "How's the weather today!"
        tts.priority = magicbot.TtsPriority.HIGH
        tts.mode = magicbot.TtsMode.CLEARTOP

        # Play speech
        status = controller.play(tts)
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to play TTS, code: %s, message: %s", status.code, status.message
            )
            return

        logging.info("Successfully played TTS")
    except Exception as e:
        logging.error("Exception occurred while playing TTS: %s", e)

def stop_tts():
    """Stop TTS playback"""
    global robot
    try:
        # Get audio controller
        controller = robot.get_audio_controller()

        # Stop speech playback
        status = controller.stop()
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to stop TTS, code: %s, message: %s", status.code, status.message
            )
            return

        logging.info("Successfully stopped TTS")
    except Exception as e:
        logging.error("Exception occurred while stopping TTS: %s", e)

def open_audio_stream():
    """Open audio stream"""

```

(下页继续)

(续上页)

```

global robot
try:
    # Get audio controller
    controller = robot.get_audio_controller()

    # Open audio stream
    status = controller.open_audio_stream()
    if status.code != magicbot.ErrorCode.OK:
        logging.error(
            "Failed to open audio stream, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

    logging.info("Successfully opened audio stream")
except Exception as e:
    logging.error("Exception occurred while opening audio stream: %s", e)

def close_audio_stream():
    """Close audio stream"""
    global robot
    try:
        # Get audio controller
        controller = robot.get_audio_controller()

        # Close audio stream
        status = controller.close_audio_stream()
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to close audio stream, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully closed audio stream")
    except Exception as e:
        logging.error("Exception occurred while closing audio stream: %s", e)

def subscribe_audio_stream():

```

(下页继续)

(续上页)

```

"""Subscribe to audio stream"""
global robot
try:
    # Get audio controller
    controller = robot.get_audio_controller()

    # Audio stream counters
    origin_counter = 0
    bf_counter = 0

    def origin_audio_callback(audio_stream):
        """Original audio stream callback function"""
        nonlocal origin_counter
        if origin_counter % 30 == 0:
            logging.info(
                "Received original audio stream data, size: %d",
                audio_stream.data_length,
            )
            origin_counter += 1

    def bf_audio_callback(audio_stream):
        """BF audio stream callback function"""
        nonlocal bf_counter
        if bf_counter % 30 == 0:
            logging.info(
                "Received BF audio stream data, size: %d", audio_stream.data_length
            )
            bf_counter += 1

    # Subscribe to audio streams
    controller.subscribe_origin_audio_stream(origin_audio_callback)
    controller.subscribe_bf_audio_stream(bf_audio_callback)

    logging.info("Subscribed to audio streams")
except Exception as e:
    logging.error("Exception occurred while subscribing to audio stream: %s", e)

def get_user_input():
    """Get user input"""
    try:
        # Python implementation of getch() on Linux systems
        import tty

```

(下页继续)

(续上页)

```
import termios

fd = sys.stdin.fileno()
old_settings = termios.tcgetattr(fd)
try:
    tty.setraw(sys.stdin.fileno())
    ch = sys.stdin.read(1)
finally:
    termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
return ch
except ImportError:
    # If termios is not available, use simple input
    return input("Please press a key: ").strip()

def main():
    """Main function"""
    global robot, running

    # Bind signal handler
    signal.signal(signal.SIGINT, signal_handler)

    logging.info("Robot model: %s", magicbot.get_robot_model())

    # Create robot instance
    robot = magicbot.MagicRobot()

    try:
        # Configure local IP address for direct network connection and initialize SDK
        local_ip = "192.168.54.111"
        if not robot.initialize(local_ip):
            logging.error("Failed to initialize robot SDK")
            robot.shutdown()
            return -1

        # Set RPC timeout to 5 seconds
        robot.set_timeout(5000)

        # Connect to robot
        status = robot.connect()
        if status.code != magicbot.ErrorCode.OK:
            logging.error(
                "Failed to connect to robot, code: %s, message: %s",
```

(下页继续)

(续上页)

```

        status.code,
        status.message,
    )
    robot.shutdown()
    return -1

logging.info("Successfully connected to robot")

# Initialize audio controller
audio_controller = robot.get_audio_controller()
if not audio_controller.initialize():
    logging.error("Failed to initialize audio controller")
    robot.disconnect()
    robot.shutdown()
    return -1

logging.info("Successfully initialized audio controller")
print_help()
logging.info("Press any key to continue (ESC to exit)...")

# Main loop
while running:
    try:
        key = get_user_input()

        if key == "\x1b": # ESC key
            break

        logging.info("Key pressed: %s", key)

        if key == "1":
            get_volume()
        elif key == "2":
            set_volume()
        elif key == "3":
            play_tts()
        elif key == "4":
            stop_tts()
        elif key == "5":
            open_audio_stream()
        elif key == "6":
            close_audio_stream()
        elif key == "7":

```

(下页继续)

(续上页)

```

        subscribe_audio_stream()
    else:
        logging.warning("Unknown key: %s", key)

        time.sleep(0.01)  # Brief delay

    except KeyboardInterrupt:
        break
    except Exception as e:
        logging.error("Exception occurred while processing user input: %s", e)
except Exception as e:
    logging.error("Exception occurred during program execution: %s", e)
return -1

finally:
    # Clean up resources
    try:
        logging.info("Clean up resources")
        # Close sensor controller
        audio_controller = robot.get_audio_controller()
        audio_controller.shutdown()
        logging.info("Audio controller closed")

        # Disconnect
        robot.disconnect()
        logging.info("Robot connection disconnected")

        # Shutdown robot
        robot.shutdown()
        logging.info("Robot shutdown")

    except Exception as e:
        logging.error("Exception occurred while cleaning up resources: %s", e)

if __name__ == "__main__":
    sys.exit(main())

```

23.4 运行说明

1. 环境准备：

```
# 设置环境变量
export PYTHONPATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$LD_LIBRARY_PATH
```

2. 运行示例：

```
# C++
./audio_example
# Python
python3 audio_example.py
```

3. 控制说明：

- ESC：退出程序
- 1：获取当前音量
- 2：设置音量为 7
- 3：播放 TTS 语音
- 4：停止 TTS 播放
- 5：打开音频流
- 6：关闭音频流
- 7：订阅音频流数据

4. 停止程序：

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

状态监控示例

该示例展示了如何使用 Magicdog-Y1 SDK 进行初始化、连接机器人、机器人状态监控 (故障、BMS) 等基本操作。

24.1 C++

示例文件: monitor_example.cpp

参考文档: monitor_reference.md

```
#include "magic_robot.h"

#include <unistd.h>
#include <csignal>

#include <iostream>

using namespace magic::y1;

magic::y1::MagicRobot robot;

void signalHandler(int signum) {
    std::cout << "Interrupt signal (" << signum << ") received.\n";

    robot.Shutdown();
}
```

(下页继续)

(续上页)

```
// 退出进程
exit(signum);
}

int main() {
    // 绑定 SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    std::string local_ip = "192.168.54.111";
    // 配置本机网线直连机器的IP地址, 并进行SDK初始化
    if (!robot.Initialize(local_ip)) {
        std::cerr << "robot sdk initialize failed." << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 设置rpc超时时间为5s
    robot.SetTimeout(5000);

    // 连接机器人
    auto status = robot.Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "connect robot failed"
            << ", code: " << status.code
            << ", message: " << status.message << std::endl;
        robot.Shutdown();
        return -1;
    }

    // 等待5s
    usleep(5000000);

    auto& monitor = robot.GetStateMonitor();

    auto state = monitor.GetCurrentState();

    std::cout << "health: " << state.bms_data.battery_health
        << ", percentage: " << state.bms_data.battery_percentage
        << ", state: " << std::to_string((int8_t)state.bms_data.battery_state)
        << ", power_supply_status: " << std::to_string((int8_t)state.bms_data.power_supply_
↵status)
        << std::endl;

    auto& faults = state.faults;
```

(下页继续)

(续上页)

```
for (auto& [code, msg] : faults) {
    std::cout << "code: " << std::to_string(code)
        << ", message: " << msg;
}

// 断开与机器人的链接
status = robot.Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed"
        << ", code: " << status.code
        << ", message: " << status.message << std::endl;
    robot.Shutdown();
    return -1;
}

robot.Shutdown();

return 0;
}
```

24.2 Python

示例文件: monitor_example.py

参考文档: monitor_reference.md

24.3 完整代码示例

```
#!/usr/bin/env python3

import sys
import time
import signal
import logging
from typing import Optional

import magicdog_y1_python as magicbot

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
```

(下页继续)

(续上页)

```

    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global variables
robot: Optional[magicbot.MagicRobot] = None

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global robot
    logging.info("Received interrupt signal (%s), exiting...", signum)
    if robot:
        robot.disconnect()
        logging.info("Robot disconnected")
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

def main():
    """Main function"""
    global robot

    # Bind signal handler
    signal.signal(signal.SIGINT, signal_handler)

    logging.info("Robot model: %s", magicbot.get_robot_model())

    # Create robot instance
    robot = magicbot.MagicRobot()

    try:
        # Configure local IP address for direct network connection and initialize SDK
        local_ip = "192.168.54.111"
        if not robot.initialize(local_ip):
            logging.error("Failed to initialize robot SDK")
            robot.shutdown()
            return -1

        # Connect to robot
        status = robot.connect()
        if status.code != magicbot.ErrorCode.OK:

```

(下页继续)

(续上页)

```

        logging.error(
            "Failed to connect to robot, code: %s, message: %s",
            status.code,
            status.message,
        )
        robot.shutdown()
        return -1

    logging.info("Successfully connected to robot")

    # Wait 5 seconds
    logging.info("Waiting 5 seconds...")
    time.sleep(5)

    # Get state monitor
    monitor = robot.get_state_monitor()

    # Get current state
    state = monitor.get_current_state()

    # Print battery information
    logging.info("Battery health: %s", state.bms_data.battery_health)
    logging.info("Battery percentage: %s%%", state.bms_data.battery_percentage)
    logging.info("Battery state: %s", state.bms_data.battery_state)
    logging.info("Power supply status: %s", state.bms_data.power_supply_status)

    # Print fault information
    faults = state.faults
    for fault in faults:
        logging.info(
            "Error code: %s, Error message: %s",
            fault.error_code,
            fault.error_message,
        )

except Exception as e:
    logging.error("Exception occurred during program execution: %s", e)
    return -1

finally:
    # Clean up resources
    try:
        logging.info("Clean up resources")
    
```

(下页继续)

(续上页)

```

        # Close state monitor
        monitor = robot.get_state_monitor()
        monitor.shutdown()

        # Disconnect
        robot.disconnect()
        logging.info("Robot connection disconnected")

        # Shutdown robot
        robot.shutdown()
        logging.info("Robot shutdown")

    except Exception as e:
        logging.error("Exception occurred while cleaning up resources: %s", e)

if __name__ == "__main__":
    sys.exit(main())

```

24.4 运行说明

1. 环境准备:

```

# 设置环境变量
export PYTHONPATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_y1_sdk/lib:$LD_LIBRARY_PATH

```

2. 运行示例:

```

# C++
./monitor_example

# Python
python3 monitor_example.py

```

3. 停止程序:

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

25.1 哪些能力已经开放？

当前文档开放主控、高层运动控制、SLAM 导航和状态监控相关说明。

25.2 哪些能力暂不开放？

传感器、音频和底层运动控制能力正在开发中，相关 API 文档和示例暂不开放。

25.3 图片无法显示怎么办？

请确认文档站的 `/image/y1/` 目录已经包含对应图片文件。

25.4 支持无线开发吗？

当前建议使用有线网络连接进行开发。将开发电脑接入机器人交换机，并把通信网卡配置到 192.168.54.X 网段，推荐使用 192.168.54.111。

25.5 SDK 接口调用报错 `Deadline Exceeded` 怎么办？

部分运动或导航接口执行时间可能超过默认 RPC 超时时间。调用这类接口时，可以根据任务耗时设置对应的 `timeout_ms` 参数。

25.6 SDK 无法订阅话题数据或话题数据不生效怎么办？

先确认本机网卡在 192.168.54.x 网段，并确认 UDP 多播路由已配置。例如本机与机器人通信的网卡为 enp0s31f6 时，运行：

```
sudo ifconfig enp0s31f6 multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev enp0s31f6
```

25.7 日志出现 Call of pthread_setschedparam with insufficient privileges! 怎么办？

这是普通用户缺少线程优先级设置权限导致的提示。可以在 /etc/security/limits.conf 中增加：

```
* - rtprio 98
```

25.8 运行时报 Invalid IP address 怎么办？

检查是否有 APP 或其他 SDK 客户端正在连接机器人。当前机器人不建议同时由多个客户端控制。

25.9 SLAM 导航不执行怎么办？

先确认已经进入定位模式和导航模式，并完成重定位。设置导航目标前，建议通过当前定位信息确认机器人位姿，再基于当前位姿设置目标点。